

Slip_1

Q1) Write a program to implement Breadth First Search (BFS) for the Monkey Banana Problem.

[10 marks]

```
from collections import deque

queue = deque(["Monkey at Door"])
visited = []

while queue:
    state = queue.popleft()
    if state not in visited:
        visited.append(state)
        print(state)
        if state == "Monkey at Door":
            queue.append("Monkey at Box")
        elif state == "Monkey at Box":
            queue.append("Box Under Banana")
        elif state == "Box Under Banana":
            queue.append("Monkey Climbs Box")
        elif state == "Monkey Climbs Box":
            queue.append("Monkey Gets Banana")
        elif state == "Monkey Gets Banana":
            print("\nGoal: Banana Obtained!")
            break
```

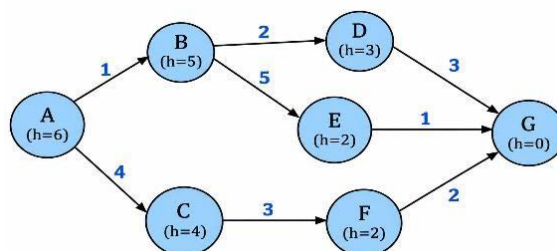
Output:

Monkey at Door
Monkey at Box
Box Under Banana
Monkey Climbs Box
Monkey Gets Banana

Goal: Banana Obtained!

Q2) Write a program to implement the A* Search Algorithm to find the shortest path between a source node and a destination node

[20 marks]



To implement A Search Algorithm*

```
from queue import PriorityQueue
```

Graph representation

```
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 3)],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}
```

Heuristic values

```
heuristic = {
    'A': 6,
```

```

'B': 5,
'C': 4,
'D': 3,
'E': 2,
'F': 2,
'G': 0
}

# A* Search Function
def a_star(start, goal):

    # OPEN list
    open_list = PriorityQueue()

    # CLOSED list
    closed = set()

    # Actual cost from start
    g_cost = {start: 0}

    # Parent of each node
    parent = {start: None}

    # Calculate f(start)
    f_cost = g_cost[start] + heuristic[start]

    # Add start node to OPEN
    open_list.put((f_cost, start))

    # Check whether goal is reached
    found = False

    # Continue until OPEN becomes empty
    while not open_list.empty():

        # Select node with smallest f(n)
        current_f, current = open_list.get()

        # If current node is goal
        if current == goal:
            found = True
            break

        # Skip if already explored
        if current in closed:
            continue

        # Move current node to CLOSED
        closed.add(current)

        # Generate successors
        for neighbor, cost in graph[current]:

            # Calculate new g(n)
            new_g = g_cost[current] + cost

            # If new path is better
            if neighbor not in g_cost or new_g < g_cost[neighbor]:

                # Update g(n)
                g_cost[neighbor] = new_g

                # Calculate f(n)
                new_f = new_g + heuristic[neighbor]

                # Store parent
                parent[neighbor] = current

            # Add node to OPEN

```

```

        open_list.put((new_f, neighbor))

    # Check whether goal was reached
    if found:

        # Create path
        path = []

        # Start from goal
        node = goal

        # Trace back to start
        while node is not None:
            path.append(node)
            node = parent[node]

        # Reverse path
        path.reverse()

        # Display result
        print("Shortest Path:", " -> ".join(path))
        print("Minimum Cost:", g_cost[goal])

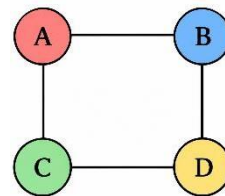
    else:
        print("No path found")

# Driver Code
Output:
source = input("Enter source node: ")
destination = input("Enter destination node: ")
a_star(source, destination)
Shortest Path: A -> B -> D -> G
Minimum Cost: 6

```

OR

Q2) Write a program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach. **[20 marks]**



```

# Program of Graph Coloring using Greedy Logic

# Function to check whether color c is safe for vertex v
def is_safe(v, graph, color, c):
    for i in range(len(graph)):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True

# Greedy graph coloring function
def graph_coloring(graph, m):
    color = [0] * len(graph)

    # Visit each vertex one by one
    for v in range(len(graph)):
        # Try colors from 1 to m

```

```

    for c in range(1, m + 1):

        if is_safe(v, graph, color, c):
            color[v] = c
            break

        # If no color is available
        if color[v] == 0:
            return None

    return color

# ----- Main Program -----

# Vertices: A, B, C, D
# Edges: A-B, A-C, B-D, C-D

graph = [
    [0, 1, 1, 0], # A
    [1, 0, 0, 1], # B
    [1, 0, 0, 1], # C
    [0, 1, 1, 0] # D
]

# Number of available colors
m = 4

vertices = ["A", "B", "C", "D"]
color_names = ["", "Red", "Blue", "Green", "Yellow"]

solution = [1, 2, 3, 4]

print("Graph Coloring Solution:")

for i in range(len(vertices)):
    print(vertices[i], "->", color_names[solution[i]])

```

Output:

Graph Coloring Solution:

A -> Red

B -> Blue

C -> Green

D -> Yellow

Slip_2

Q1) Write a program to solve the Water Jug Problem using BFS.

[10 marks]

2. Write a program to solve the Water Jug Problem using BFS

```
from collections import deque

queue = deque(["Both Jugs Empty"])

visited = []

while queue:
    state = queue.popleft()

    if state not in visited:
        visited.append(state)
        print(state)

        if state == "Both Jugs Empty":
            queue.append("Fill 3-Litre Jug")

        elif state == "Fill 3-Litre Jug":
            queue.append("pour jug 3 liter into 4 liter")

        elif state == "pour jug 3 liter into 4 liter":
            queue.append("fill 3 liter jug again")

        elif state == "fill 3 liter jug again":
            queue.append("pour 3 liter into 4 liter")

        elif state == "pour 3 liter into 4 liter":
            queue.append("Get 2 Litre in 4 litre")

        elif state == "Get 2 Litre in 4 litre":
            print("\nGoal: Required Water Obtained!")
            break
```

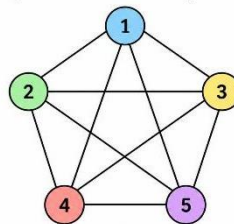
Output:

Both Jugs Empty
Fill 3-Litre Jug
pour jug 3 liter into 4 liter
fill 3 liter jug again
pour 3 liter into 4 liter
Get 2 Litre in 4 litre

Goal: Required Water Obtained!

Q2) Write a program to implement the Backtracking Algorithm for solving a constraint satisfaction problem.

[20 marks]



Program of Graph Coloring using Backtracking

Function to check whether color can be assigned

```
def is_safe(vertex, graph, color, c):
```

```

    for i in range(5):

        if graph[vertex][i] == 1 and color[i] == c:
            return False

    return True

# Backtracking function
def graph_coloring(graph, color, vertex):

    # If all vertices are colored
    if vertex == 5:
        return True

    # Try all colors
    for c in range(1, 6):

        if is_safe(vertex, graph, color, c):

            # Assign color
            color[vertex] = c

            # Move to next vertex
            if graph_coloring(graph, color, vertex + 1):
                return True

            # Backtrack
            color[vertex] = 0

    return False

# ----- Main Program -----

# Adjacency matrix
graph = [
    [0, 1, 1, 1, 1], # 1
    [1, 0, 1, 1, 1], # 2
    [1, 1, 0, 1, 1], # 3
    [1, 1, 1, 0, 1], # 4
    [1, 1, 1, 1, 0] # 5
]

# Color array
# 0 means no color assigned
color = [0, 0, 0, 0, 0]

# Color names
color_names = ["", "Blue", "Green", "Yellow", "Red", "Purple"]

# Start coloring from vertex 1
if graph_coloring(graph, color, 0):

    print("Graph Coloring Solution:")

    for i in range(5):
        print(i + 1, "->", color_names[color[i]])

    else:
        print("No solution exists.")
Graph Coloring Solution:
1 -> Blue
2 -> Green
3 -> Yellow
4 -> Red
5 -> Purple

```

OR

Q2) Write a program to implement a Game Tree Representation for a two- [20 marks]
player game.

Write a program to implement a Game Tree Representation for a two-player game.

Program for Minimax Algorithm

```
import math
```

```
# Minimax Function
```

```
def minimax(depth, node_index, is_max, values, max_depth):
```

```
    # Leaf node
```

```
    if depth == max_depth:
```

```
        return values[node_index]
```

```
    if is_max:
```

```
        # MAX player
```

```
        left = minimax(depth + 1, node_index * 2, False, values, max_depth)
```

```
        right = minimax(depth + 1, node_index * 2 + 1, False, values, max_depth)
```

```
        return max(left, right)
```

```
    else:
```

```
        # MIN player
```

```
        left = minimax(depth + 1, node_index * 2, True, values, max_depth)
```

```
        right = minimax(depth + 1, node_index * 2 + 1, True, values, max_depth)
```

```
        return min(left, right)
```

```
# ----- Main Program -----
```

```
# Leaf values
```

```
leaf_values = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
# Tree depth
```

```
tree_depth = int(math.log2(len(leaf_values)))
```

```
# Find optimal value
```

```
best_score = minimax(0, 0, True, leaf_values, tree_depth)
```

```
print("Leaf Node Values:", leaf_values)
```

```
print("Optimal value selected by Minimax:", best_score)
```

Output:

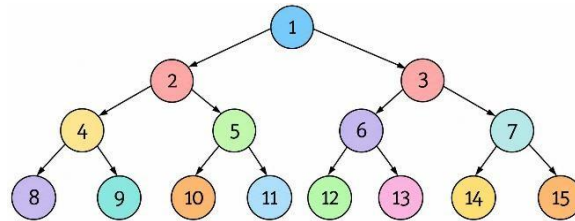
Leaf Node Values: [3, 5, 2, 9, 12, 5, 23, 23]

Optimal value selected by Minimax: 12

Slip_3

Q1) Write a program to implement BFS traversal for a graph.

[10 marks]



BFS Traversal of a Graph

```
from collections import deque
```

Graph represented as an adjacency list

```
graph = {                                # Input data.
```

```
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [8, 9],
    5: [10, 11],
    6: [12, 13],
    7: [14, 15],
    8: [],
    9: [],
    10: [],
    11: [],
    12: [],
    13: [],
    14: [],
    15: []
}
```

Starting node

```
start_node = 1                        # start_node → Specifies from which node BFS will begin.
```

Function to perform BFS Traversal

```
def bfs(graph, start):                # def bfs(...) → Defines the algorithm.
```

```
    visited = []
    queue = deque([start])
```

```
    visited.append(start)
```

```
    while queue:                       # while queue: → Executes the traversal.
```

```
        node = queue.popleft()
```

```
        print(node, end=" ")
```

```
        for neighbour in graph[node]:
```

```
            if neighbour not in visited:
```

```
                visited.append(neighbour)
```

```
                queue.append(neighbour)
```

```
print("BFS Traversal:")
```

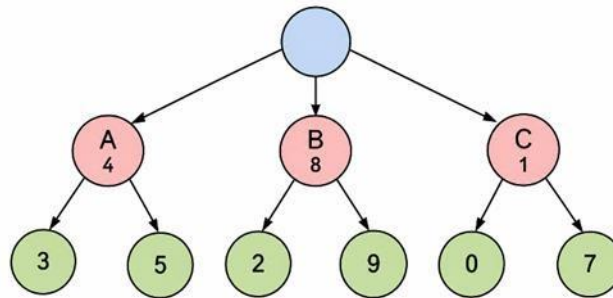
```
bfs(graph, start_node)      # bfs(graph, start_node) → Performs BFS traversal.
```

Output:

BFS Traversal:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

- Q2)** Write a program to implement Alpha-Beta Pruning for optimizing the Minimax search process. **[20 marks]**



Write a program to implement Alpha-Beta Pruning for optimizing the Minimax search process.

```
import math
```

Alpha-Beta Pruning function

```
def alpha_beta(node, depth, alpha, beta, is_max, tree):
```

If leaf node is reached

```
if depth == 0:
    return node
```

```
if is_max:
```

MAX player's turn

```
value = -math.inf
```

```
for child in tree[node]:
```

```
    value = max(value, alpha_beta(child, depth - 1, alpha, beta, False, tree))
```

```
    alpha = max(alpha, value)
```

Beta Cutoff

```
if alpha >= beta:
    break
```

```
return value
```

```
else:
```

MIN player's turn

```
value = math.inf
```

```
for child in tree[node]:
```

```
    value = min(value, alpha_beta(child, depth - 1, alpha, beta, True, tree))
```

```
    beta = min(beta, value)
```

Alpha Cutoff

```
if alpha >= beta:
    break
```

```
return value
```

Game tree

```
tree = {
```

```

    "Root": ["A", "B", "C"],
    "A": [3, 5],
    "B": [2, 9],
    "C": [0, 7]
}

# Find the optimal value
best_score = alpha_beta( "Root",
    2,
    -math.inf,
    math.inf,
    True,
    tree
)

print("Leaf Node Values:", [3, 5, 2, 9, 0, 7])
print("Optimal value selected by Alpha-Beta Pruning:", best_score)

```

Output:

Leaf Node Values: [3, 5, 2, 9, 0, 7]

Optimal value selected by Alpha-Beta Pruning: 3

OR

Q2) Write a program to implement Propositional Logic and evaluate logical expressions using operators such as AND, OR, and NOT. **[20 marks]**

Program for Propositional Logic

Facts

P = True

Q = False

Logical AND function

```
def logical_and(p, q):
    return p and q
```

Logical OR function

```
def logical_or(p, q):
    return p or q
```

Logical NOT function

```
def logical_not(p):
    return not p
```

Test the logical expressions

```
print("P:", P)
```

```
print("Q:", Q)
```

```
if logical_and(P, Q):
    print("P AND Q is True.")
else:
    print("P AND Q is False.")
```

```
if logical_or(P, Q):
    print("P OR Q is True.")
else:
    print("P OR Q is False.")
```

```
if logical_not(P):
    print("NOT P is True.")
else:
    print("NOT P is False.")
```

Output:
P: True
Q: False
P AND Q is False.
P OR Q is True.
NOT P is False.

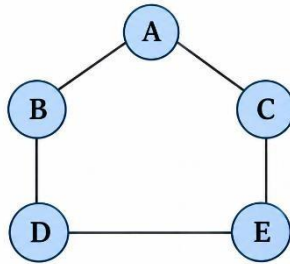
Q3) Viva

[05 marks]

Slip_4

Q1) Write a program to solve a state-space search problem using BFS.
Start Vertex = A

[10 marks]



Write a program to solve a state-space search problem using BFS.

Import deque to implement the Queue (FIFO)
from collections **import** deque

Function to perform Breadth First Search (BFS)
def bfs(graph, start):

Create a queue and insert the starting node
queue = deque([start])

List to store visited nodes
visited = []

Continue searching until the queue becomes empty
while queue:

Delete the first node from the queue
node = queue.popleft()

Process the node only if it is not visited
if node **not in** visited:

Mark the node as visited
visited.append(node)

Display the node
print(node, end=" ")

Explore all neighbouring nodes
for neighbour **in** graph[node]:

Add unvisited neighbour to the queue
if neighbour **not in** visited:
queue.append(neighbour)

State Space Representation of the Graph
graph = {
 'A': ['B', 'C'],
 'B': ['A', 'D'],
 'C': ['A', 'E'],
 'D': ['B', 'E'],
 'E': ['C', 'D']
}

Define the Start Vertex
start = 'A'

Call the BFS function
print("BFS Traversal:")
bfs(graph, start)

Output:
BFS Traversal:
A B C D E

Q2) Write a program to implement Predicate Logic using predicates, variables, and quantifier. **[20 marks]**

```
# Program for Predicate Logic

# Facts
cats = ["Tom", "Kitty"]

# Predicate function
def is_cat(animal):
    # Check whether the animal is a cat
    return animal in cats

# Rule function
def is_animal(animal):
    # A cat is an animal
    return is_cat(animal)

# Test with Tom
name = "Tom"

# Display the name
print("Name:", name)

# Check whether Tom is a cat
if is_cat(name):
    print(name, "is a Cat.")

# Check whether Tom is an animal
if is_animal(name):
    print(name, "is an Animal.")
else:
    print(name, "is not found.")

# Check all cats
print("\nAll Cats:")

# Check each cat in the list
for cat in cats:
    # Check whether the cat is an animal
    if is_animal(cat):
        print(cat, "is an Animal.")
```

Output:
Name: Tom
Tom is a Cat.
Tom is an Animal.

All Cats:
Tom is an Animal.
Kitty is an Animal.

OR

- Q2)** Write a program to implement Gaussian NB classifier with 3 classes on numeric data, use make_blobs to create a dataset X of 200 sample points in 2D Space, 3 clusters and each cluster spread (SD) over 1.5. For testing (Prediction), use the following values to test the Gaussian NB Classifier. **[20 marks]**

Input (Values) : [-2, 5], [0,0], [6, -0.3]

Output : [0 1 1]

Print the accuracy score using the accuracy_score() from sklearn.metrics library..

python -m pip install scikit-learn -> nstall this lib first in cell

Gaussian NB Classifier

```
from sklearn.datasets import make_blobs
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

Create training data

```
X, y = make_blobs(
    n_samples=200,
    centers=3,
    n_features=2,
    cluster_std=1.5,
    random_state=42
)
```

Create and train the Gaussian NB model

```
model = GaussianNB()
model.fit(X, y)
```

Test data

```
test_data = [
    [-2, 5],
    [0, 0],
    [6, -0.3]
]
```

Make prediction

```
prediction = model.predict(test_data)
```

Display prediction

```
print("Prediction:", prediction)
```

Calculate accuracy

```
y_pred = model.predict(X)
accuracy = accuracy_score(y, y_pred)
```

Display accuracy

```
print("Accuracy:", accuracy)
```

Output:

Prediction: [0 1 1]

Accuracy: 1.0

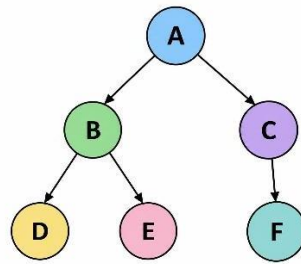
- Q3)** Viva

[05 marks]

Slip_5

Q1) Write a program to implement Depth Limited Search (DLS).

[10 marks]



Depth Limited Search (DLS) Program

Function to perform Depth Limited Search

```
def dls(graph, node, goal, limit, path):
```

Add the current node to the path

```
path.append(node)
```

Check whether the goal node is reached

```
if node == goal:
```

```
    return path
```

Stop searching if the depth limit is reached

```
if limit == 0:
```

```
    path.pop()
```

```
    return None
```

Explore all neighbouring nodes

```
for neighbour in graph[node]:
```

Perform DLS recursively with reduced depth limit

```
result = dls(graph, neighbour, goal, limit - 1, path)
```

Return the path if the goal node is found

```
if result:
```

```
    return result
```

Backtrack if the goal is not found

```
path.pop()
```

```
return None
```

State Space Representation of the Graph

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
```

Define the Initial State, Goal State and Depth Limit

```
start = 'A'
```

```
goal = 'F'
```

```
depth_limit = 2
```

Call the DLS Function

```
result = dls(graph, start, goal, depth_limit, [])
```

Display the Result

```
if result:
    print("Path Found :", " -> ".join(result))
else:
    print("Goal State Not Found Within Depth Limit.")
```

Output:
Path Found : A -> C -> F

Q2) Write a program to implement Bernoulli NB classifier with 2 classes on numeric data, use make_classification to create a dataset X of 300 sample points in 2D space with the following parameters :

n_features=2, n_informative=2, n_redundant=0, n_classes=2.

For testing (Prediction), use the following values to test the Bernoulli NB Classifier.

Input (Values) : [[0, 0], [0, 1], [1, 0], [1, 1]]

Output : [0, 1, 0, 1]

Program to implement Bernoulli Naive Bayes Classifier

```
from sklearn.datasets import make_classification
from sklearn.naive_bayes import BernoulliNB
```

Create dataset

```
X, y = make_classification(
    n_samples=300,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_classes=2,
    random_state=0
)
```

Create and train Bernoulli NB classifier

```
model = BernoulliNB()
model.fit(X, y)
```

Input values for testing

```
test_data = [
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
]
```

Predict the class

```
prediction = model.predict(test_data)
```

Display input and output

```
print("Input:", test_data)
print("Output:", prediction)
```

Output:

Input: [[0, 0], [0, 1], [1, 0], [1, 1]]

Output: [0 1 0 1]

OR

Q2) Write a program to implement a Support Vector Machine (SVM) classifier on the Iris dataset. **[20 marks]**

Input (Dataset and Class Labels):

```
X = [ [5.1, 3.5, 1.4, 0.2], [4.9, 3.0, 1.4, 0.2], [4.7, 3.2, 1.3, 0.2], [4.6, 3.1, 1.5, 0.2], [5.0, 3.6, 1.4, 0.2], [7.0, 3.2, 4.7, 1.4], [6.4, 3.2, 4.5, 1.5], [6.9, 3.1, 4.9, 1.5], [6.3, 3.3, 6.0, 2.5], [5.8, 2.7, 5.1, 1.9] ]
```

```
y = [ 0, 0, 0, 0, 0, 1, 1, 1, 2, 2 ]
```

```
X_test (Test Data) = [ [5.2, 3.4, 1.5, 0.2], [6.5, 3.0, 4.6, 1.5], [6.2, 3.0, 5.2, 2.0] ]
```

```
from sklearn import svm
```

```
X = [  
    [5.1, 3.5, 1.4, 0.2],  
    [4.9, 3.0, 1.4, 0.2],  
    [4.7, 3.2, 1.3, 0.2],  
    [4.6, 3.1, 1.5, 0.2],  
    [5.0, 3.6, 1.4, 0.2],  
    [7.0, 3.2, 4.7, 1.4],  
    [6.4, 3.2, 4.5, 1.5],  
    [6.9, 3.1, 4.9, 1.5],  
    [6.3, 3.3, 6.0, 2.5],  
    [5.8, 2.7, 5.1, 1.9]  
]
```

```
y = [0, 0, 0, 0, 0, 1, 1, 1, 2, 2]
```

```
model = svm.SVC(kernel='linear')  
model.fit(X, y)
```

```
X_test = [  
    [5.2, 3.4, 1.5, 0.2],  
    [6.5, 3.0, 4.6, 1.5],  
    [6.2, 3.0, 5.2, 2.0]  
]
```

```
prediction = model.predict(X_test)
```

```
print("Test Data:", X_test)  
print("Prediction:", prediction)
```

Output:

Test Data: [[5.2, 3.4, 1.5, 0.2], [6.5, 3.0, 4.6, 1.5], [6.2, 3.0, 5.2, 2.0]]

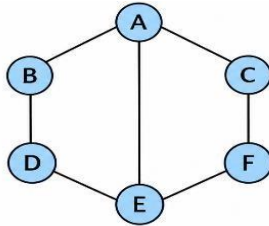
Prediction: [0 1 2]

Q3) Viva **[05 marks]**

Slip_6

Q1) Write a program to implement DFS traversal for a graph.

[10 marks]



Function to perform DFS

```
def dfs(graph, node, visited):
```

```
    # Mark the current node as visited
    visited.append(node)
```

```
    # Print the visited node
    print(node, end=" ")
```

```
    # Visit all unvisited neighbours
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(graph, neighbour, visited)
```

Graph representation using a dictionary

```
graph = {
    'A': ['B', 'E', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'F'],
    'D': ['B', 'E'],
    'E': ['A', 'D', 'F'],
    'F': ['C', 'E']
}
```

```
# List to store visited nodes
visited = []
```

```
# Starting node
start = 'A'
```

```
print("DFS Traversal:")
```

```
dfs(graph, start, visited)
```

Output:

```
DFS Traversal:
A B D E F C
```

Q2) Write a program to implement a Non-Linear Support Vector Machine (SVM) using the Radial Basis Function (RBF) Kernel.

[20 marks]

Input Data :

```
X = [ [1, 2], [2, 3], [5, 5] ]
```

Class Labels :

```
y = [0, 0, 1]
```

Test Input : [4, 4]

```
import sys
import subprocess
```

```
subprocess.check_call([sys.executable, "-m", "pip", "install", "matplotlib"])
```

Output:

0

```
import matplotlib.pyplot as plt
```

```
print("Matplotlib is working")
```

Output:

Matplotlib is working

```
from sklearn import svm
import matplotlib.pyplot as plt
import numpy as np
```

```
X = [
    [1, 2],
    [2, 3],
    [5, 5]
]
```

```
y = [0, 0, 1]
```

```
model = svm.SVC(kernel='rbf')
model.fit(X, y)
```

```
test_input = [[4, 4]]
```

```
prediction = model.predict(test_input)
```

```
if prediction[0] == 1:
    print("Prediction: Class 1")
else:
    print("Prediction: Class 0")
```

```
# Create graph
```

```
x_min = 0
x_max = 6
y_min = 1
y_max = 6
```

```
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 200),
    np.linspace(y_min, y_max, 200)
)
```

```
# Predict graph points
```

```
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

```
# Show decision regions
```

```
plt.contourf(xx, yy, Z, alpha=0.3)
```

```
# Show training data
```

```
for i in range(len(X)):
    if y[i] == 0:
        plt.scatter(X[i][0], X[i][1],
                    marker='o', s=100, label='Class 0')
    else:
        plt.scatter(X[i][0], X[i][1],
                    marker='s', s=100, label='Class 1')
```

```
# Show test input
```

```
plt.scatter(
    test_input[0][0],
    test_input[0][1],
    marker='*',
    s=250,
```

```

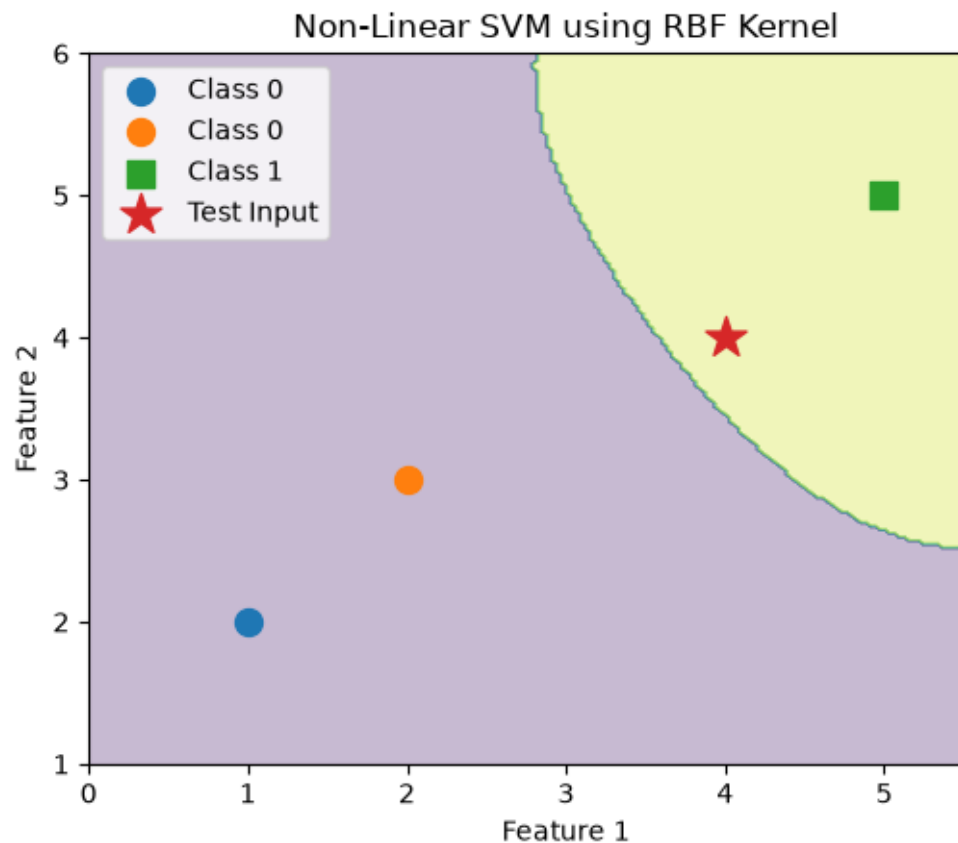
label="Test Input"
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Non-Linear SVM using RBF Kernel")
plt.legend()
plt.show()

```

Output:

Prediction: Class 1



OR

Q2) Write a program to implement a Basic Artificial Neural Network (ANN) **[20 marks]**

for predicting outputs from input data as given below. Use different values for the hyperparameters for the learning rate and number of epochs. Find the ideal value for learning rate and # of epochs.

Learning rate: 0.1, and number of epoch = 20 (initially).

Input_vectors = [[3, 1.5], [2, 1], [4, 1.5], [3, 4], [3.5, 0.5], [2, 0.5],
[5.5, 1], [1, 1]]

and targets = [0, 1, 0, 1, 0, 1, 1, 0]

```
from sklearn.neural_network import MLPClassifier
```

```
# Training data
```

```
X = [
    [3, 1.5],
    [2, 1],
    [4, 1.5],
    [3, 4],
    [3.5, 0.5],
    [2, 0.5],
    [5.5, 1],

```

```

    [1, 1]
]

# Target values
y = [0, 1, 0, 1, 0, 1, 1, 0]

# Create and train the model
model = MLPClassifier(
    hidden_layer_sizes=(5,),
    learning_rate_init=0.1,
    max_iter=20,
    random_state=42
)

model.fit(X, y)

# New input data
new_input = [[4, 2]]

# Predict
prediction = model.predict(new_input)

print("Prediction:", prediction)

Output:
Prediction: [0]

```

Q3) Viva

[05 marks]

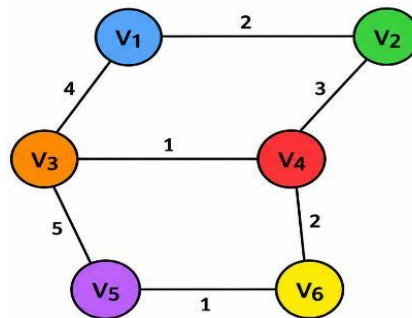
Slip_7

Q1) Write a program to implement the A* Search Algorithm to find the shortest path between a source node and a destination node.

[10 marks]

Source = V_1

Destination = V_6



To implement A Search Algorithm*

```
from queue import PriorityQueue
```

Graph representation

```
graph = {
    'V1': [('V2', 2), ('V3', 4)],
    'V2': [('V1', 2), ('V4', 3)],
    'V3': [('V1', 4), ('V4', 1), ('V5', 5)],
    'V4': [('V2', 3), ('V3', 1), ('V6', 2)],
    'V5': [('V3', 5), ('V6', 1)],
    'V6': []
}
```

Heuristic values

No heuristic values are given in the diagram

```
heuristic = {
    'V1': 0,
    'V2': 0,
    'V3': 0,
    'V4': 0,
    'V5': 0,
    'V6': 0
}
```

A Search Function*

```
def a_star(start, goal):
```

OPEN list

```
open_list = PriorityQueue()
```

CLOSED list

```
closed = set()
```

Actual cost from start

```
g_cost = {start: 0}
```

Parent of each node

```
parent = {start: None}
```

Calculate f(start)

```
f_cost = g_cost[start] + heuristic[start]
```

Add start node to OPEN

```
open_list.put((f_cost, start))
```

Check whether goal is reached

```
found = False
```

Continue until OPEN becomes empty

```

while not open_list.empty():

    # Select node with smallest f(n)
    current_f, current = open_list.get()

    # If current node is goal
    if current == goal:
        found = True
        break

    # Skip if already explored
    if current in closed:
        continue

    # Move current node to CLOSED
    closed.add(current)

    # Generate successors
    for neighbor, cost in graph[current]:

        # Calculate new g(n)
        new_g = g_cost[current] + cost

        # If new path is better
        if neighbor not in g_cost or new_g < g_cost[neighbor]:

            # Update g(n)
            g_cost[neighbor] = new_g

            # Calculate f(n)
            new_f = new_g + heuristic[neighbor]

            # Store parent
            parent[neighbor] = current

            # Add node to OPEN
            open_list.put((new_f, neighbor))

    # Check whether goal was reached
    if found:

        # Create path
        path = []

        # Start from goal
        node = goal

        # Trace back to start
        while node is not None:
            path.append(node)
            node = parent[node]

        # Reverse path
        path.reverse()

        # Display result
        print("Shortest Path:", " -> ".join(path))
        print("Minimum Cost:", g_cost[goal])

    else:
        print("No path found")

# Driver Code
source = input("Enter source node: ")
destination = input("Enter destination node: ")

a_star(source, destination)

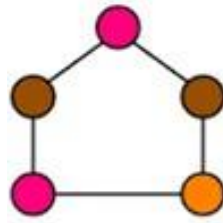
```

Ouput:

Shortest Path: V1 -> V2 -> V4 -> V6

Minimum Cost: 7

- Q2)** Write a program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach. **[20 marks]**



*# Program to solve Map Coloring Problem using CSP
Using Backtracking*

Function to check whether color can be assigned

```
def is_safe(vertex, graph, color, c):
```

```
    for i in range(5):
        if graph[vertex][i] == 1 and color[i] == c:
            return False
```

```
    return True
```

Backtracking function

```
def graph_coloring(graph, color, vertex):
```

If all vertices are colored

```
if vertex == 5:
    return True
```

Try different colors

```
for c in range(1, 4):
```

Check whether color is safe

```
if is_safe(vertex, graph, color, c):
```

Assign color

```
color[vertex] = c
```

Move to next vertex

```
if graph_coloring(graph, color, vertex + 1):
    return True
```

Backtrack

```
color[vertex] = 0
```

```
return False
```

----- Main Program -----

Graph connections:

1 -> 2, 3

2 -> 1, 4

3 -> 1, 5

4 -> 2, 5

5 -> 3, 4

```
graph = [
```

```

    [0, 1, 1, 0, 0], # 1
    [1, 0, 0, 1, 0], # 2
    [1, 0, 0, 0, 1], # 3
    [0, 1, 0, 0, 1], # 4
    [0, 0, 1, 1, 0] # 5
]

# Number of colors
m = 3

# 0 means no color assigned
color = [0, 0, 0, 0, 0]

# Color names
color_names = ["", "Pink", "Brown", "Orange"]

# Apply graph coloring
if graph_coloring(graph, color, 0):

    print("Map Coloring Solution:")

    for i in range(5):
        print(i + 1, "->", color_names[color[i]])

else:
    print("No solution exists with", m, "colors.")

```

Output:

Map Coloring Solution:

```

1 -> Pink
2 -> Brown
3 -> Brown
4 -> Pink
5 -> Orange

```

OR

Q2) Write a program to develop a Tic-Tac-Toe Game in which the computer uses the Minimax Algorithm to play optimally. **[20 marks]**

Program for Tic-Tac-Toe using Minimax Algorithm

```

import math

# Display the board
def print_board(board):
    print()
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
    print()

# Check whether a player has won
def check_winner(board, player):
    winning_positions = [
        [0, 1, 2],
        [3, 4, 5],
        [6, 7, 8],
        [0, 3, 6],
        [1, 4, 7],
        [2, 5, 8],
        [0, 4, 8],
        [2, 4, 6]
    ]

```

```

    for position in winning_positions:
        if (board[position[0]] == player and
            board[position[1]] == player and
            board[position[2]] == player):
            return True

    return False

# Check whether board is full
def is_full(board):
    return " " not in board

# Recursive Minimax function
def minimax(board, is_max):

    # Computer wins
    if check_winner(board, "O"):
        return 1

    # Human wins
    if check_winner(board, "X"):
        return -1

    # Draw
    if is_full(board):
        return 0

    if is_max:
        # MAX player's turn
        best_score = -math.inf

        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                score = minimax(board, False)
                board[i] = " "
                best_score = max(best_score, score)

        return best_score

    else:
        # MIN player's turn
        best_score = math.inf

        for i in range(9):
            if board[i] == " ":
                board[i] = "X"
                score = minimax(board, True)
                board[i] = " "
                best_score = min(best_score, score)

        return best_score

# Find the best move for computer
def find_best_move(board):
    best_score = -math.inf
    best_move = -1

    for i in range(9):
        if board[i] == " ":
            board[i] = "O"
            score = minimax(board, False)
            board[i] = " "

            if score > best_score:
                best_score = score

```

```

        best_move = i

    return best_move

# ----- Main Program -----

# Empty Tic-Tac-Toe board
board = [" "] * 9

print("TIC-TAC-TOE GAME")
print("You = X")
print("Computer = O")

print_board(board)

# Game starts
while True:

    # Human move
    move = int(input("Enter your position (1-9): ")) - 1

    if move < 0 or move > 8 or board[move] != " ":
        print("Invalid move. Try again.")
        continue

    board[move] = "X"
    print_board(board)

    # Check human winner
    if check_winner(board, "X"):
        print("You Win!")
        break

    # Check draw
    if is_full(board):
        print("Game Draw!")
        break

    # Computer move using Minimax
    computer_move = find_best_move(board)
    board[computer_move] = "O"

    print("Computer chooses position:", computer_move + 1)
    print_board(board)

    # Check computer winner
    if check_winner(board, "O"):
        print("Computer Wins!")
        break

    # Check draw
    if is_full(board):
        print("Game Draw!")
        break

```

Output:

TIC-TAC-TOE GAME

You = X

Computer = O

```

| |
--+---+--
| |
--+---+--
| |

```

```

X | |

```

```
--+---+--  
| |  
--+---+--  
| |
```

Computer chooses position: 5

```
X | |  
--+---+--  
| O |  
--+---+--  
| |
```

```
X | X |  
--+---+--  
| O |  
--+---+--  
| |
```

Computer chooses position: 3

```
X | X | O  
--+---+--  
| O |  
--+---+--  
| |
```

Q3) Viva

[05 marks]

Slip_8

Q1) Write a program to solve the 4-Puzzle Problem using the A* Search Algorithm. **[10 marks]**

Write a program to solve the 4-Puzzle Problem using A Search Algorithm*

Import PriorityQueue

from queue **import** PriorityQueue

Calculate heuristic value

h(n) = number of misplaced tiles

def heuristic(state, goal):

count = 0

Check all 4 positions

for i **in** range(4):

Ignore the blank space (0)

if state[i] != 0 **and** state[i] != goal[i]:

count += 1

return count

Display the puzzle

def display(state):

print(state[0], state[1])

print(state[2], state[3])

print()

Generate possible next states

def get_neighbors(state):

neighbors = []

Find the blank space

zero = state.index(0)

Possible positions of the blank space

moves = []

Move blank UP

if zero >= 2:

moves.append(zero - 2)

Move blank DOWN

if zero < 2:

moves.append(zero + 2)

Move blank LEFT

if zero % 2 != 0:

moves.append(zero - 1)

Move blank RIGHT

if zero % 2 != 1:

moves.append(zero + 1)

Create new states

for move **in** moves:

new_state = list(state)

```

        # Swap blank with the tile
        new_state[zero], new_state[move] = \
            new_state[move], new_state[zero]

        # Convert list into tuple
        neighbors.append(tuple(new_state))

    return neighbors

# A* Search Algorithm
def AStar(start, goal):

    # Create OPEN and CLOSED
    OPEN = PriorityQueue()
    CLOSED = set()

    # g(Start) = 0
    g_cost = {start: 0}

    # Store parent of each state
    parent = {start: None}

    # Calculate f(Start)
    #  $f(n) = g(n) + h(n)$ 
    f = g_cost[start] + heuristic(start, goal)

    # Put Start into OPEN
    OPEN.put((f, start))

    # Repeat until OPEN becomes empty
    while not OPEN.empty():

        # Select node with smallest f(n)
        f, current = OPEN.get()

        # Check whether goal is reached
        if current == goal:
            break

        # Skip if already explored
        if current in CLOSED:
            continue

        # Add current node to CLOSED
        CLOSED.add(current)

        # Generate successors
        for neighbor in get_neighbors(current):

            # Skip already explored states
            if neighbor in CLOSED:
                continue

            # Calculate g(n)
            new_g = g_cost[current] + 1

            # Check for a better path
            if neighbor not in g_cost or new_g < g_cost[neighbor]:

                # Update g(n)
                g_cost[neighbor] = new_g

                # Calculate h(n)
                h = heuristic(neighbor, goal)

                # Calculate f(n)

```

```

        f = new_g + h

        # Store parent
        parent[neighbor] = current

        # Add neighbor to OPEN
        OPEN.put((f, neighbor))

    # Check if goal is found
    if goal in g_cost:

        # Create solution path
        path = []

        node = goal

        # Trace path from goal to start
        while node is not None:

            path.append(node)
            node = parent[node]

        # Reverse path
        path.reverse()

        print("Solution Found!")
        print("Number of moves:", len(path) - 1)

        # Display every step
        for i, state in enumerate(path):

            print("Step", i)
            display(state)

    else:

        print("No solution found")

# Starting state
start = (
    1, 2,
    0, 3
)

# Goal state
goal = (
    1, 2,
    3, 0
)

# Call A* Algorithm
AStar(start, goal)

Output:
Solution Found!
Number of moves: 1
Step 0
1 2
0 3

Step 1
1 2
3 0

```

Q2) Write a program or report that compares Minimax and Alpha-Beta Pruning on the same game tree. **[20 marks]**

Program to compare Minimax and Alpha-Beta Pruning

```
import math
```

```
# ----- Minimax Function -----
```

```
def minimax(depth, node_index, is_max, values, max_depth):
```

```
    # If leaf node is reached
```

```
    if depth == max_depth:
```

```
        return values[node_index]
```

```
    if is_max:
```

```
        # MAX player's turn
```

```
        left = minimax(
```

```
            depth + 1,
```

```
            node_index * 2,
```

```
            False,
```

```
            values,
```

```
            max_depth
```

```
        )
```

```
        right = minimax(
```

```
            depth + 1,
```

```
            node_index * 2 + 1,
```

```
            False,
```

```
            values,
```

```
            max_depth
```

```
        )
```

```
        return max(left, right)
```

```
    else:
```

```
        # MIN player's turn
```

```
        left = minimax(
```

```
            depth + 1,
```

```
            node_index * 2,
```

```
            True,
```

```
            values,
```

```
            max_depth
```

```
        )
```

```
        right = minimax(
```

```
            depth + 1,
```

```
            node_index * 2 + 1,
```

```
            True,
```

```
            values,
```

```
            max_depth
```

```
        )
```

```
        return min(left, right)
```

```
# ----- Alpha-Beta Function -----
```

```
def alpha_beta(depth, node_index, is_max, values, max_depth, alpha, beta):
```

```
    # If leaf node is reached
```

```

if depth == max_depth:
    return values[node_index]

if is_max:

    # MAX player's turn
    left = alpha_beta(
        depth + 1,
        node_index * 2,
        False,
        values,
        max_depth,
        alpha,
        beta
    )

    value = left
    alpha = max(alpha, value)

    # Beta Cutoff
    if alpha >= beta:
        return value

    right = alpha_beta(
        depth + 1,
        node_index * 2 + 1,
        False,
        values,
        max_depth,
        alpha,
        beta
    )

    value = max(value, right)
    alpha = max(alpha, value)

    return value

else:

    # MIN player's turn
    left = alpha_beta(
        depth + 1,
        node_index * 2,
        True,
        values,
        max_depth,
        alpha,
        beta
    )

    value = left
    beta = min(beta, value)

    # Alpha Cutoff
    if alpha >= beta:
        return value

    right = alpha_beta(
        depth + 1,
        node_index * 2 + 1,
        True,
        values,
        max_depth,
        alpha,
        beta
    )

```

```

        value = min(value, right)
        beta = min(beta, value)

    return value

# ----- Main Program -----

# Leaf node values of the game tree
leaf_values = [3, 5, 2, 9, 12, 5, 23, 23]

# Height of tree
tree_depth = int(math.log2(len(leaf_values)))

# Find the optimal value using Minimax
minimax_score = minimax(
    depth=0,
    node_index=0,
    is_max=True,
    values=leaf_values,
    max_depth=tree_depth
)

# Find the optimal value using Alpha-Beta Pruning
alpha_beta_score = alpha_beta(
    depth=0,
    node_index=0,
    is_max=True,
    values=leaf_values,
    max_depth=tree_depth,
    alpha=-math.inf,
    beta=math.inf
)

# Display results
print("Leaf Node Values:", leaf_values)

print("Optimal value using Minimax:", minimax_score)

print("Optimal value using Alpha-Beta Pruning:", alpha_beta_score)

if minimax_score == alpha_beta_score:
    print("Both algorithms give the same optimal value.")

```

Output:

```

Leaf Node Values: [3, 5, 2, 9, 12, 5, 23, 23]
Optimal value using Minimax: 12
Optimal value using Alpha-Beta Pruning: 12
Both algorithms give the same optimal value.

```

OR

Q2) Write a program to design a Rule-Based Expert System for simple decision-making. **[20 marks]**

Program of Rule-Based Expert System for Student Result

```

# Input from user
marks = float(input("Enter marks: "))
attendance = float(input("Enter attendance percentage: "))

```

```

# Rule 1: Check whether student has passed
if marks >= 40:
    passed = True
else:
    passed = False

# Rule 2: Check attendance
if attendance >= 75:
    good_attendance = True
else:
    good_attendance = False

# Rule 3 and Rule 4: Decision and recommendation
if passed and good_attendance:
    result = "Eligible for Exam"
    recommendation = "Student has good marks and attendance."
elif passed and not good_attendance:
    result = "Attendance Shortage"
    recommendation = "Improve attendance before the examination."
elif not passed and good_attendance:
    result = "Failed"
    recommendation = "Improve marks and prepare better."
else:
    result = "Not Eligible"
    recommendation = "Improve marks and attendance."

# Output
print("\n--- Student Report ---")
print("Passed:", passed)
print("Good Attendance:", good_attendance)
print("Decision:", result)
print("Recommendation:", recommendation)

Output:
--- Student Report ---
Passed: True
Good Attendance: True
Decision: Eligible for Exam
Recommendation: Student has good marks and attendance.

```

Q3) Viva

[05 marks]

Slip_9

Q1) Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function.

```
# Hill Climbing Algorithm
# Find the maximum value of a simple objective function

# Define the heuristic values
heuristic = {
    'A': 15,
    'B': 25,
    'D': 30,
    'C': 35,
    'E': 50
}

# Define the graph
graph = {
    'A': ['B', 'D'],
    'B': ['A', 'C', 'E'],
    'D': ['A', 'C'],
    'C': ['D', 'B', 'E'],
    'E': ['C', 'B']
}

# Define the Hill Climbing function
def hill_climbing(start):

    # Set the starting state as the current state
    current = start

    # Store the path
    path = [current]

    # Repeat the process until no better neighbor is found
    while True:

        # Generate neighboring states
        neighbors = graph[current]

        # Select the neighbor having the highest value
        best_neighbor = max(
            neighbors,
            key=lambda x: heuristic[x]
        )

        # Check whether the best neighbor is better
        # than the current state
        if heuristic[best_neighbor] <= heuristic[current]:

            # If there is no improvement, stop the algorithm
            break

        # Move to the better neighboring state
        current = best_neighbor

        # Add the state to the path
        path.append(current)

    # Return the final state and path
    return path, heuristic[current]
```

```

# Set the initial state
start = 'A'

# Call the Hill Climbing function
path, maximum_value = hill_climbing(start)

# Display the starting state
print("Starting State:", start)

# Display the path followed
print("Path:", " -> ".join(path))

# Display the state where maximum value is found
print("Maximum State:", path[-1])

# Display the maximum heuristic value
print("Maximum Value:", maximum_value)

```

Output:

```

Starting State: A
Path: A -> D -> C -> E
Maximum State: E
Maximum Value: 50

```

Q2) Write a program to implement the Forward Chaining inference mechanism. **[20 marks]**

Program to implement Forward Chaining Inference Mechanism

```

# Known facts
facts = ["Ravi is a Student"]

# IF-THEN Rules
rules = [
    ("Ravi is a Student", "Ravi studies AI"),
    ("Ravi studies AI", "Ravi learns Machine Learning"),
    ("Ravi learns Machine Learning", "Ravi can get a Job")
]

# Desired goal
goal = "Ravi can get a Job"

print("Initial Facts:", facts)

# Forward Chaining
while True:

    rule_fired = False

    # Compare facts with rules
    for condition, conclusion in rules:

        # IF condition is true
        if condition in facts and conclusion not in facts:

            # THEN add new fact
            facts.append(conclusion)

```

```

    print("New Fact:", conclusion)

    rule_fired = True

    # Stop when goal is reached
    if conclusion == goal:
        break

    # Check whether goal is reached
    if goal in facts:
        break

    # Stop if no rule can be applied
    if rule_fired == False:
        break

# Display final result
print("\nFinal Facts:", facts)

if goal in facts:
    print("Goal Reached:", goal)
else:
    print("Goal Not Reached")

```

Output:

Initial Facts: ['Ravi is a Student']
 New Fact: Ravi studies AI
 New Fact: Ravi learns Machine Learning
 New Fact: Ravi can get a Job

Final Facts: ['Ravi is a Student', 'Ravi studies AI', 'Ravi learns Machine Learning', 'Ravi can get a Job']
 Goal Reached: Ravi can get a Job

OR

- Q2)** Write a program to implement the Decision Tree Classifier for a classification problem. Use the dataset provided with columns: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, Outcome(is the class label / target variable: 0 or 1). **[20 marks]**

Pregnancies	Glucose	BP	SkinThick	Insulin	BMI	DiabetesPF	Age	Outcome
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1
5	116	74	0	0	25.6	0.201	30	0
3	78	50	32	88	31	0.248	26	1
10	115	0	0	0	35.3	0.134	29	0
2	197	70	45	543	30.5	0.158	53	1
8	125	96	0	0	0	0.232	54	1
4	110	92	0	0	37.6	0.191	30	0

Split the dataset into 70:30 – train:test.

Evaluate the model using `accuracy_score()`. Optimize the model's performance by tuning the criterion (default is GINI, use entropy) and `max_depth` (=2, 3 or 4) parameter of the DT Classifier

Program to implement Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
```

Dataset

```
X = [
    [6, 148, 72, 35, 0, 33.6, 0.627, 50],
    [1, 85, 66, 29, 0, 26.6, 0.351, 31],
    [8, 183, 64, 0, 0, 23.3, 0.672, 32],
    [1, 89, 66, 23, 94, 28.1, 0.167, 21],
    [0, 137, 40, 35, 168, 43.1, 2.288, 33],
    [3, 78, 50, 32, 88, 31.0, 0.248, 26],
    [10, 115, 0, 0, 0, 35.3, 0.134, 29],
    [2, 197, 70, 45, 0, 30.5, 0.158, 53],
    [4, 110, 92, 0, 0, 37.6, 0.191, 54],
    [5, 125, 80, 30, 120, 32.0, 0.300, 40]
]
```

Target variable

```
y = [1, 0, 1, 0, 1, 0, 0, 1, 0, 1]
```

Split dataset into 70% training and 30% testing

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42
)
```

Decision Tree using GINI

```

model = DecisionTreeClassifier(
    criterion='gini',
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, prediction)

print("Actual Values:", y_test)
print("Predicted Values:", prediction)
print("Accuracy using GINI:", accuracy)

# Display clean Decision Tree
feature_names = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age"
]

plt.figure(figsize=(20, 12))

plot_tree(
    model,
    feature_names=feature_names,
    class_names=["0", "1"],
    filled=True,
    rounded=True,
    fontsize=10,
    max_depth=3
)

plt.title(
    "Decision Tree Classifier - GINI Criterion",
    fontsize=18
)

plt.tight_layout()
plt.show()

# Model Optimization
print("\n--- Model Optimization ---")

for depth in [2, 3, 4]:

    model = DecisionTreeClassifier(
        criterion='entropy',

```

```

    max_depth=depth,
    random_state=42
)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

accuracy = accuracy_score(y_test, prediction)

print("Criterion: entropy, Max Depth:", depth)
print("Accuracy:", accuracy)

```

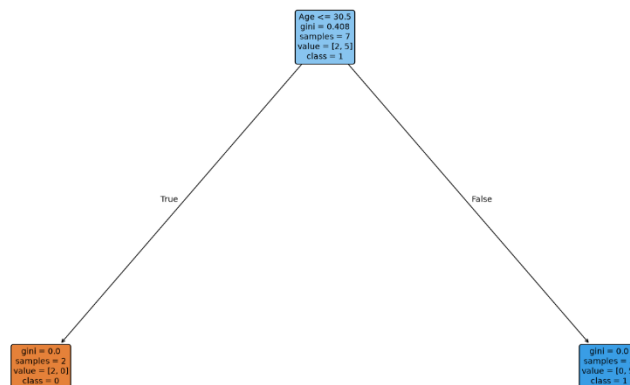
Output:

Actual Values: [0, 0, 0]

Predicted Values: [1 1 0]

Accuracy using GINI: 0.3333333333333333

Decision Tree Classifier - GINI Criterion



--- Model Optimization ---

Criterion: entropy, Max Depth: 2

Accuracy: 0.3333333333333333

Criterion: entropy, Max Depth: 3

Accuracy: 0.3333333333333333

Criterion: entropy, Max Depth: 4

Accuracy: 0.3333333333333333

Q3) Viva

**[05
marks]**

Slip_10

Q1) Write a program to implement Means-End Analysis for solving a goal-based problem

To implement Means-End Analysis

Problem: Help the robot reach the GOAL (Room D)

from START (Room A) by reducing the difference step by step.

Rooms and their connections

```
rooms = {  
    'A': ['B'],  
    'B': ['C'],  
    'C': ['D'],  
    'D': []  
}
```

```
def means_end_analysis(start, goal):
```

```
    current_state = start
```

```
    print("MEANS-END ANALYSIS")  
    print("Robot starts from Room", start)  
    print("Goal is Room", goal)  
    print()
```

```
    while current_state != goal:
```

```
        # Find the difference between Current State and Goal
```

```
        difference = ord(goal) - ord(current_state)
```

```
        print("Current State:", current_state)  
        print("Goal State:", goal)  
        print("Difference:", difference, "room(s) away")
```

```
        # Select an action that reduces the difference
```

```
        next_room = rooms[current_state][0]
```

```
        print("Action: Move to next adjacent room")
```

```
        # Apply the action
```

```
        current_state = next_room
```

```
        # Update Current State
```

```
        print("New State:", current_state)  
        print()
```

```
    # Goal reached
```

```
    print("Current State:", current_state)  
    print("Goal State:", goal)  
    print("Difference: 0 (No difference)")  
    print("Action: Stop")  
    print("Goal Reached!")  
    print()  
    print("Path Found: Room A -> Room B -> Room C -> Room D")
```

Output:

```
# Driver code
```

```
start_room = 'A'
```

```
goal_room = 'D'
```

```
means_end_analysis(start_room, goal_room)
```

```
MEANS-END ANALYSIS
```

```
Robot starts from Room A
```

```
Goal is Room D
```

```
Current State: A
```

```
Goal State: D
```

Difference: 3 room(s) away
Action: Move to next adjacent room
New State: B

Current State: B
Goal State: D
Difference: 2 room(s) away
Action: Move to next adjacent room
New State: C

Current State: C
Goal State: D
Difference: 1 room(s) away
Action: Move to next adjacent room
New State: D

Current State: D
Goal State: D
Difference: 0 (No difference)
Action: Stop
Goal Reached!

Path Found: Room A -> Room B -> Room C -> Room D

Q2) Write a program to implement the Random Forest Classifier for classification tasks.

```
dataset = { 'Study_Hours': [2, 3, 4, 5, 6, 7, 1, 2, 5, 8],
            'Attendance': [60, 65, 70, 75, 80, 85, 55, 62, 78, 90],
            'Assignment_Score': [55, 60, 65, 70, 75, 80, 50, 58, 72, 85],
            'Result': ['Fail', 'Fail', 'Pass', 'Pass', 'Pass',
                       'Pass', 'Fail', 'Fail', 'Pass', 'Pass'] }
```

Cell-1:

```
import sys
print(sys.executable)
```

Cell-2:

```
%pip install pandas scikit-learn
```

Cell-3:

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
```

```
print("Installation successful")
```

Output:

Installation successful

Cell-4 :

```
# Program: Random Forest Classifier
# Student Result Prediction
```

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
```

```
# Dataset
```

```
dataset = {
    'Study_Hours': [2,3,4,5,6,7,1,2,5,8],
    'Attendance': [60,65,70,75,80,85,55,62,78,90],
    'Assignment_Score': [55,60,65,70,75,80,50,58,72,85],
    'Result': ['Fail','Fail','Pass','Pass','Pass',
               'Pass','Fail','Fail','Pass','Pass']
}
```

```
# Convert dataset into DataFrame
```

```
df = pd.DataFrame(dataset)
```

```

# Input features
X = df[['Study_Hours', 'Attendance', 'Assignment_Score']]

# Output
y = df['Result']

# Create Random Forest model
model = RandomForestClassifier(n_estimators=5, random_state=42)

# Train the model
model.fit(X, y)

# Test data
test_data = [[6, 82, 78]]

# Predict result
prediction = model.predict(test_data)

# Display output
print("Study Hours:", test_data[0][0])
print("Attendance:", test_data[0][1])
print("Assignment Score:", test_data[0][2])
print("Prediction:", prediction[0])

```

Output:

```

Study Hours: 6
Attendance: 82
Assignment Score: 78
Prediction: Pass

```

OR

- Q2)** Write a program to demonstrate hyperplane classification using a Support Vector Machine on the iris dataset. Create a Pipeline containing a StandardScaler and a LinearSVC (with $c = 1$ and use hinge loss). **[20 marks]**

No.	Sepal Length	Sepal Width	Petal Length	Petal Width
1	5.1	3.5	1.4	0.2
2	4.9	3.0	1.4	0.2
3	4.7	3.2	1.3	0.2
4	4.6	3.1	1.5	0.2
5	5.0	3.6	1.4	0.2
6	7.0	3.2	4.7	1.4
7	6.4	3.2	4.5	1.5
8	6.9	3.1	4.9	1.5
9	5.5	2.3	4.0	1.3
10	6.5	2.8	4.6	1.5

Program: Hyperplane Classification using SVM

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC
import matplotlib.pyplot as plt
import numpy as np

```

```

X = [
    [5.1, 3.5, 1.4, 0.2],
    [4.9, 3.0, 1.4, 0.2],
    [4.7, 3.2, 1.3, 0.2],
    [4.6, 3.1, 1.5, 0.2],
    [5.0, 3.6, 1.4, 0.2],
    [7.0, 3.2, 4.7, 1.4],
    [6.4, 3.2, 4.5, 1.5],
    [6.9, 3.1, 4.9, 1.5],
    [5.5, 2.3, 4.0, 1.3],
    [6.5, 2.8, 4.6, 1.5]
]

y = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]

model = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', LinearSVC(C=1, loss='hinge', random_state=42))
])

model.fit(X, y)

test_flower = [[6.0, 3.0, 4.5, 1.5]]

prediction = model.predict(test_flower)

if prediction[0] == 1:
    print("Prediction: Versicolor")
else:
    print("Prediction: Setosa")

# Hyperplane graph
X_graph = np.array(X)

plt.scatter(X_graph[:5, 0], X_graph[:5, 1], label="Setosa")
plt.scatter(X_graph[5:, 0], X_graph[5:, 1], label="Versicolor")

svm_model = model.named_steps['svm']
scaler = model.named_steps['scaler']

w = svm_model.coef_[0]
b = svm_model.intercept_[0]

x = np.linspace(4, 8, 100)

x_scaled = (x - scaler.mean_[0]) / scaler.scale_[0]

y_scaled = -(w[0] * x_scaled + b) / w[1]

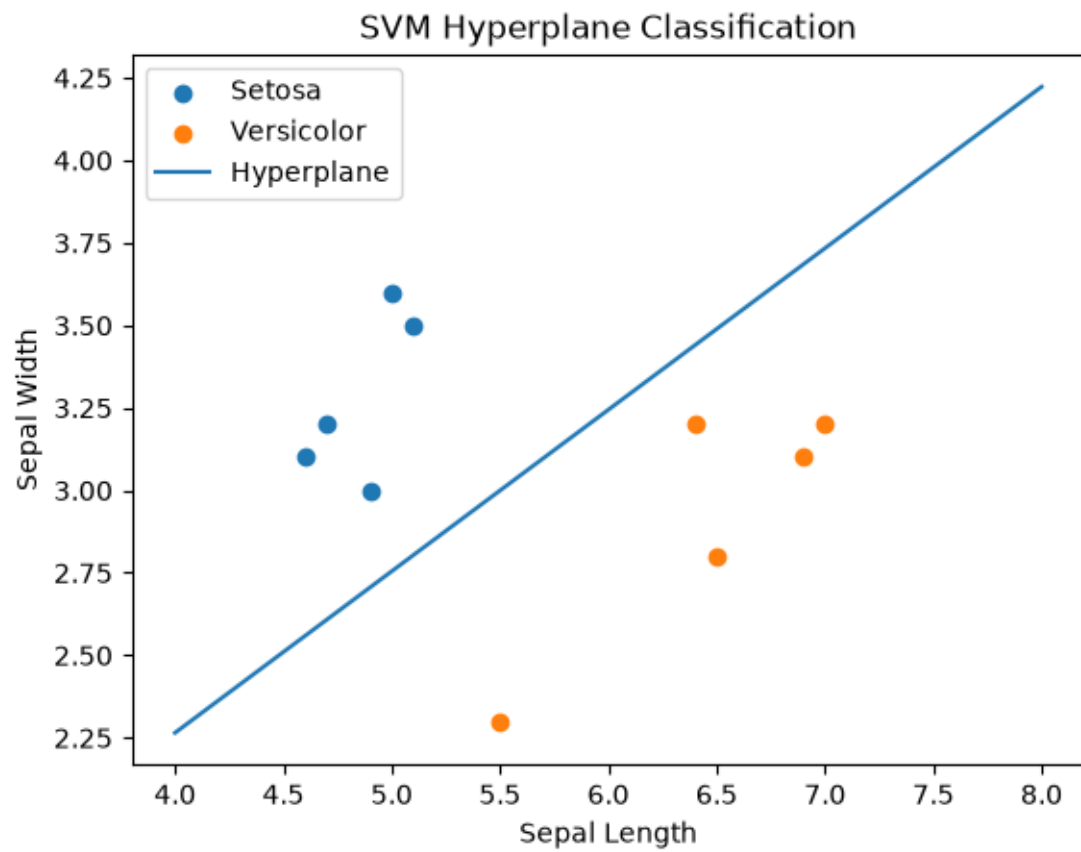
y_graph = y_scaled * scaler.scale_[1] + scaler.mean_[1]

plt.plot(x, y_graph, label="Hyperplane")

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("SVM Hyperplane Classification")
plt.legend()
plt.show()

# OUTPUT
# Prediction: Versicolor
Prediction: Versicolor

```



Q3) Viva

[05 marks]

Slip_11

Q1) Write a program or report that compares Minimax and Alpha-Beta Pruning on the same game tree.

Program to compare Minimax and Alpha-Beta Pruning

```
import math
```

```
# ----- Minimax Function -----
```

```
def minimax(depth, node_index, is_max, values, max_depth):
```

```
    # If leaf node is reached
```

```
    if depth == max_depth:
```

```
        return values[node_index]
```

```
    if is_max:
```

```
        # MAX player's turn
```

```
        left = minimax(
```

```
            depth + 1,
```

```
            node_index * 2,
```

```
            False,
```

```
            values,
```

```
            max_depth
```

```
        )
```

```
        right = minimax(
```

```
            depth + 1,
```

```
            node_index * 2 + 1,
```

```
            False,
```

```
            values,
```

```
            max_depth
```

```
        )
```

```
        return max(left, right)
```

```
    else:
```

```
        # MIN player's turn
```

```
        left = minimax(
```

```
            depth + 1,
```

```
            node_index * 2,
```

```
            True,
```

```
            values,
```

```
            max_depth
```

```
        )
```

```
        right = minimax(
```

```
            depth + 1,
```

```
            node_index * 2 + 1,
```

```
            True,
```

```
            values,
```

```
            max_depth
```

```
        )
```

```
        return min(left, right)
```

```
# ----- Alpha-Beta Function -----
```

```
def alpha_beta(depth, node_index, is_max, values, max_depth, alpha, beta):
```

```
    # If leaf node is reached
```

```
    if depth == max_depth:
```

```

    return values[node_index]

if is_max:

    # MAX player's turn
    left = alpha_beta(
        depth + 1,
        node_index * 2,
        False,
        values,
        max_depth,
        alpha,
        beta
    )

    value = left
    alpha = max(alpha, value)

    # Beta Cutoff
    if alpha >= beta:
        return value

    right = alpha_beta(
        depth + 1,
        node_index * 2 + 1,
        False,
        values,
        max_depth,
        alpha,
        beta
    )

    value = max(value, right)
    alpha = max(alpha, value)

    return value

else:

    # MIN player's turn
    left = alpha_beta(
        depth + 1,
        node_index * 2,
        True,
        values,
        max_depth,
        alpha,
        beta
    )

    value = left
    beta = min(beta, value)

    # Alpha Cutoff
    if alpha >= beta:
        return value

    right = alpha_beta(
        depth + 1,
        node_index * 2 + 1,
        True,
        values,
        max_depth,
        alpha,
        beta
    )

    value = min(value, right)

```

```

        beta = min(beta, value)

    return value

# ----- Main Program -----

# Leaf node values of the game tree
leaf_values = [3, 5, 2, 9, 12, 5, 23, 23]

# Height of tree
tree_depth = int(math.log2(len(leaf_values)))

# Find the optimal value using Minimax
minimax_score = minimax(
    depth=0,
    node_index=0,
    is_max=True,
    values=leaf_values,
    max_depth=tree_depth
)

# Find the optimal value using Alpha-Beta Pruning
alpha_beta_score = alpha_beta(
    depth=0,
    node_index=0,
    is_max=True,
    values=leaf_values,
    max_depth=tree_depth,
    alpha=-math.inf,
    beta=math.inf
)

# Display results
print("Leaf Node Values:", leaf_values)

print("Optimal value using Minimax:", minimax_score)

print("Optimal value using Alpha-Beta Pruning:", alpha_beta_score)

if minimax_score == alpha_beta_score:
    print("Both algorithms give the same optimal value.")

```

Output:

```

Leaf Node Values: [3, 5, 2, 9, 12, 5, 23, 23]
Optimal value using Minimax: 12
Optimal value using Alpha-Beta Pruning: 12
Both algorithms give the same optimal value.

```

Q2) Write a program to implement the Backward Chaining inference mechanism. **[20 marks]**

```
# Program to implement Backward Chaining
# Rule-Based Medical Diagnosis

# Input from user
temperature = float(input("Enter body temperature (°C): "))
cough = input("Does the patient have a cough? (yes/no): ").lower()

# Facts
if temperature > 38:
    fever = True
else:
    fever = False

# Goal: Patient has Flu
goal = "Flu"

# Backward Chaining
# To prove Flu, we need Fever AND Cough

if goal == "Flu":
    # Check first condition: Fever
    if fever:
        # Check second condition: Cough
        if cough == "yes":
            result = "Patient has Flu"
        else:
            result = "Flu cannot be proved because patient has no cough."
    else:
        result = "Flu cannot be proved because patient has no fever."

# Output
print("\n--- Backward Chaining Result ---")
print("Goal:", goal)
print("Fever:", fever)
print("Cough:", cough)
print("Result:", result)
```

Output:

--- Backward Chaining Result ---

Goal: Flu

Fever: False

Cough: no

Result: Flu cannot be proved because patient has no fever.

OR

- Q2)** Write a program to implement the Random Forest Classifier for classification tasks. Use the dataset provided: car_evaluation.csv (7 columns/features and 10 rows). All the features are of categorical data type, given by buying, maint, doors, persons, lug_boot, safety and class. Class is the target variable. Rename column names as ['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'class']. Check the frequency counts of categorical variables. Check for missing values. Drop 'class' feature to create X and let y = 'class' variable. Split the dataset as 67:33% - train:test. Encode all the categorical variables. Implement the RF classifier with default parameters. Display the accuracy of the RF classifier. **[20 marks]**

No.	buying	maint	doors	persons	lug_boot	safety	class
1	vhigh	vhigh	2	2	small	low	unacc
2	vhigh	vhigh	2	2	small	med	unacc
3	vhigh	vhigh	2	2	small	high	unacc
4	vhigh	vhigh	2	2	med	low	unacc
5	vhigh	vhigh	2	2	med	med	unacc
6	vhigh	vhigh	2	2	med	high	unacc
7	vhigh	vhigh	2	2	big	low	unacc
8	vhigh	vhigh	2	2	big	med	unacc
9	vhigh	vhigh	2	2	big	high	unacc
10	vhigh	vhigh	2	4	small	low	unacc

Import csv file from google or github - car_evaluation.csv

Program: Random Forest Classifier
Car Evaluation Dataset

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

Read CSV file
df = pd.read_csv("car_evaluation.csv")

Rename columns
df.columns = ['buying', 'maint', 'doors', 'persons',
 'lug_boot', 'safety', 'class']

Display dataset
print("Dataset:")
print(df)

Check frequency counts of categorical variables
print("\nFrequency Counts:")

```
for column in df.columns:
    print("\n", column)
    print(df[column].value_counts())
```

Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

Drop class column to create X
X = df.drop('class', axis=1)

Create y as class variable
y = df['class']

```

# Encode categorical variables
for column in X.columns:
    encoder = LabelEncoder()
    X[column] = encoder.fit_transform(X[column])

# Encode target variable
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

# Split dataset into 67% training and 33% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42
)

# Create Random Forest Classifier
model = RandomForestClassifier()

# Train the model
model.fit(X_train, y_train)

# Predict test data
prediction = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, prediction)

# Display accuracy
print("\nRandom Forest Classifier Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")

```

Output:

Dataset:

	buying	maint	doors	persons	lug_boot	safety	class
0	vhigh	vhigh	2	2	small	med	unacc
1	vhigh	vhigh	2	2	small	high	unacc
2	vhigh	vhigh	2	2	med	low	unacc
3	vhigh	vhigh	2	2	med	med	unacc
4	vhigh	vhigh	2	2	med	high	unacc
...	...	...	...	...	...	...	...
1722	low	low	5more	more	med	med	good
1723	low	low	5more	more	med	high	vgood
1724	low	low	5more	more	big	low	unacc
1725	low	low	5more	more	big	med	good
1726	low	low	5more	more	big	high	vgood

[1727 rows x 7 columns]

Frequency Counts:

```

buying
buying
high    432
med     432
low     432
vhigh   431
Name: count, dtype: int64

```

maint

```
maint
high  432
med   432
low   432
vhigh 431
Name: count, dtype: int64
```

```
doors
doors
3     432
4     432
5more 432
2     431
Name: count, dtype: int64
```

```
persons
persons
4     576
more  576
2     575
Name: count, dtype: int64
```

```
lug_boot
lug_boot
med   576
big   576
small 575
Name: count, dtype: int64
```

```
safety
safety
med   576
high  576
low   575
Name: count, dtype: int64
```

```
class
class
unacc 1209
acc    384
good   69
vgood   65
Name: count, dtype: int64
```

```
Missing Values:
buying    0
maint     0
doors     0
persons   0
lug_boot  0
safety    0
class     0
dtype: int64
```

Random Forest Classifier Accuracy: 0.9578947368421052
Accuracy Percentage: 95.78947368421052 %

Q3) Viva

[05 marks]

Slip_12

Q1) Write a program to implement Breadth First Search (BFS) for the Monkey Banana Problem.

[10 marks]

```
from collections import deque

queue = deque(["Monkey at Door"])
visited = []

while queue:
    state = queue.popleft()
    if state not in visited:
        visited.append(state)
        print(state)
        if state == "Monkey at Door":
            queue.append("Monkey at Box")
        elif state == "Monkey at Box":
            queue.append("Box Under Banana")
        elif state == "Box Under Banana":
            queue.append("Monkey Climbs Box")
        elif state == "Monkey Climbs Box":
            queue.append("Monkey Gets Banana")
        elif state == "Monkey Gets Banana":
            print("\nGoal: Banana Obtained!")
            break
```

Output:

Monkey at Door
Monkey at Box
Box Under Banana
Monkey Climbs Box
Monkey Gets Banana

Goal: Banana Obtained!

Q2) Write a program or report to compare Linear SVM and Non-Linear SVM (RBF Kernel). **[20 marks]**

X = [[1, 2], [2, 3], [3, 1], [4, 2], [5, 5], [6, 6], [7, 4], [8, 5]]

Y = [0, 0, 0, 0, 1, 1, 1, 1]

Program to Compare Linear SVM and Non-Linear SVM (RBF Kernel)

```
from sklearn import svm
import matplotlib.pyplot as plt
import numpy as np
```

Input data

```
X = [
    [1, 2],
    [2, 3],
    [3, 1],
    [4, 2],
    [5, 5],
    [6, 6],
    [7, 4],
    [8, 5]
]
```

Output classes

```
Y = [0, 0, 0, 0, 1, 1, 1, 1]
```

```

# Test data
test_data = [[6, 5]]

# Linear SVM
linear_model = svm.SVC(kernel='linear')
linear_model.fit(X, Y)

linear_prediction = linear_model.predict(test_data)

print("Linear SVM Prediction:", linear_prediction[0])

# Non-Linear SVM using RBF Kernel
rbf_model = svm.SVC(kernel='rbf')
rbf_model.fit(X, Y)

rbf_prediction = rbf_model.predict(test_data)

print("RBF SVM Prediction:", rbf_prediction[0])

# Convert data to NumPy array
X = np.array(X)
Y = np.array(Y)

# Create graphs
plt.figure(figsize=(12, 5))

# Linear SVM Graph
plt.subplot(1, 2, 1)

plt.scatter(X[Y == 0, 0], X[Y == 0, 1], label="Class 0")
plt.scatter(X[Y == 1, 0], X[Y == 1, 1], label="Class 1")

# Linear decision boundary
w = linear_model.coef_[0]
b = linear_model.intercept_[0]

x_line = np.linspace(0, 9, 100)
y_line = -(w[0] * x_line + b) / w[1]

plt.plot(x_line, y_line, label="Decision Boundary")

# Test point
plt.scatter(
    test_data[0][0],
    test_data[0][1],
    marker="*",
    s=200,
    label="Test Point"
)

plt.title("Linear SVM")
plt.xlabel("X1")
plt.ylabel("X2")
plt.legend()
plt.grid(True)

# RBF SVM Graph
plt.subplot(1, 2, 2)

plt.scatter(X[Y == 0, 0], X[Y == 0, 1], label="Class 0")
plt.scatter(X[Y == 1, 0], X[Y == 1, 1], label="Class 1")

# Create grid
xx, yy = np.meshgrid(
    np.linspace(0, 9, 300),
    np.linspace(0, 7, 300)
)

grid = np.c_[xx.ravel(), yy.ravel()]

```

```

# Predict grid points
Z = rbf_model.predict(grid)
Z = Z.reshape(xx.shape)

# RBF decision boundary
plt.contour(xx, yy, Z, levels=[0.5])

# Test point
plt.scatter(
    test_data[0][0],
    test_data[0][1],
    marker="*",
    s=200,
    label="Test Point"
)

plt.title("Non-Linear SVM (RBF Kernel)")
plt.xlabel("X1")
plt.ylabel("X2")
plt.legend()
plt.grid(True)

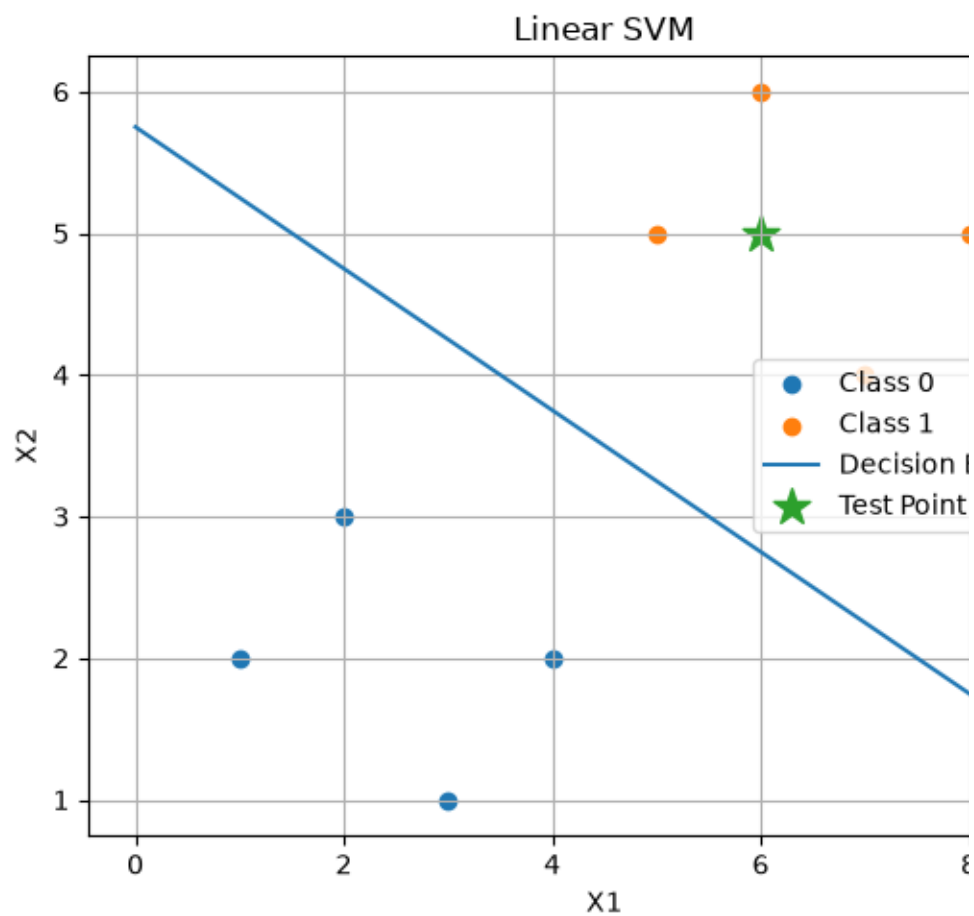
plt.tight_layout()
plt.show()

```

Output:

Linear SVM Prediction: 1

RBF SVM Prediction: 1



OR

Q2) Write a program to implement an Artificial Neural Network (ANN) for a Linear Regression problem. **[20 marks]**

Program to implement ANN for Linear Regression
Problem: Predict House Price

```
from sklearn.neural_network import MLPRegressor

# Training data
X = [
    [500],
    [800],
    [1000],
    [1200],
    [1500]
]

# House price in Lakhs
y = [20, 32, 40, 48, 60]

# Create and train ANN model
model = MLPRegressor(
    hidden_layer_sizes=(5,),
    max_iter=5000,
    random_state=42
)

model.fit(X, y)

# New house data
new_house = [[1800]]

# Predict house price
prediction = model.predict(new_house)

print("House Area:", new_house[0][0], "sq.ft")
print("Predicted House Price:", round(prediction[0], 2), "Lakhs")
```

Output:

House Area: 1800 sq.ft

Predicted House Price: 73.32 Lakhs

Q3) Viva **[05 marks]**

Slip_13

Q1) Write a program to solve the Water Jug Problem using BFS.

[10 marks]

2. Write a program to solve the Water Jug Problem using BFS

```
from collections import deque

queue = deque(["Both Jugs Empty"])

visited = []

while queue:
    state = queue.popleft()

    if state not in visited:
        visited.append(state)
        print(state)

        if state == "Both Jugs Empty":
            queue.append("Fill 3-Litre Jug")

        elif state == "Fill 3-Litre Jug":
            queue.append("pour jug 3 liter into 4 liter")

        elif state == "pour jug 3 liter into 4 liter":
            queue.append("fill 3 liter jug again")

        elif state == "fill 3 liter jug again":
            queue.append("pour 3 liter into 4 liter")

        elif state == "pour 3 liter into 4 liter":
            queue.append("Get 2 Litre in 4 litre")

        elif state == "Get 2 Litre in 4 litre":
            print("\nGoal: Required Water Obtained!")
            break
```

Output:

Both Jugs Empty
Fill 3-Litre Jug
pour jug 3 liter into 4 liter
fill 3 liter jug again
pour 3 liter into 4 liter
Get 2 Litre in 4 litre

Goal: Required Water Obtained!

Q2) Write a program to implement a Voting Classifier by combining multiple machine learning models such as Logistic Regression and Decision Tree.

[20 marks]

Loan Approval System

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import VotingClassifier
```

```
# Training data
# Features: [Income_High, CreditScore_Good]
# Income_High: 1 = Yes, 0 = No
# CreditScore_Good: 1 = Yes, 0 = No
```

```
X = [
    [1, 1], # High income, Good credit
```

```

    [1, 0], # High income, Poor credit
    [0, 1], # Low income, Good credit
    [0, 0] # Low income, Poor credit
]

# Target values
# 1 = Approve Loan
# 0 = Reject Loan

y = [1, 0, 0, 0]

# Create Logistic Regression model
logistic_model = LogisticRegression()

# Create Decision Tree model
tree_model = DecisionTreeClassifier(random_state=42)

# Create Voting Classifier
model = VotingClassifier(
    estimators=[
        ('logistic', logistic_model),
        ('tree', tree_model)
    ],
    voting='soft'
)

# Train the model
model.fit(X, y)

# Test data
# Example: High income and Good credit
test_data = [[1, 1]]

# Make prediction
prediction = model.predict(test_data)

# Display result
if prediction[0] == 1:
    print("Loan Approved")
else:
    print("Loan Rejected")

# OUTPUT
# Loan Approved
Loan Approved

```

OR

Q2) Write a program to develop a simple Knowledge Graph representing relationships among entities. **[20 marks]**

*# Program to develop a simple Knowledge Graph
representing relationships among entities*

```

facts = [
    ("Rahul", "studies", "Computer Science"),
    ("Rahul", "livesIn", "Pune"),
    ("Pune", "locatedIn", "Maharashtra"),
    ("Priya", "studies", "Computer Science"),
    ("Priya", "livesIn", "Pune")
]

# Display all relationships
print("Knowledge Graph Relationships:")

```

```

for subject, relation, obj in facts:
    print(subject, relation, obj)

# Find relationships of Rahul
person = "Rahul"

print("\nRelationships of", person, ":")

for subject, relation, obj in facts:
    if subject == person:
        print(person, relation, obj)

# Find location of Rahul
city = ""

for subject, relation, obj in facts:
    if subject == person and relation == "livesIn":
        city = obj

# Find state of the city
state = ""

for subject, relation, obj in facts:
    if subject == city and relation == "locatedIn":
        state = obj

# Display inferred relationship
if city != "" and state != "":
    print("\nKnowledge Graph Inference:")
    print(person, "livesIn", city)
    print(city, "locatedIn", state)
    print(person, "livesInState", state)
else:
    print("Relationship not found.")

```

Output:

Knowledge Graph Relationships:
 Rahul studies Computer Science
 Rahul livesIn Pune
 Pune locatedIn Maharashtra
 Priya studies Computer Science
 Priya livesIn Pune

Relationships of Rahul :
 Rahul studies Computer Science
 Rahul livesIn Pune

Knowledge Graph Inference:
 Rahul livesIn Pune
 Pune locatedIn Maharashtra
 Rahul livesInState Maharashtra

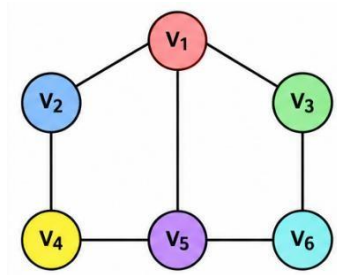
Q3) Viva

[05 marks]

Slip_14

Q1) Write a program to implement BFS traversal for a graph.

[10 marks]



BFS Traversal of a Graph

```
from collections import deque
```

Graph represented as an adjacency list

```
graph = {                                # Input data.
```

```
    1: [2, 3, 5],
```

```
    2: [1, 4],
```

```
    3: [1, 6],
```

```
    4: [2, 5],
```

```
    5: [1, 4, 6],
```

```
    6: [3, 5]
```

```
}
```

Starting node

```
start_node = 1                        # start_node → Specifies from which node BFS will begin.
```

Function to perform BFS Traversal

```
def bfs(graph, start):                # def bfs(...) → Defines the algorithm.
```

```
    visited = []
```

```
    queue = deque([start])
```

```
    visited.append(start)
```

```
    while queue:                       # while queue: → Executes the traversal.
```

```
        node = queue.popleft()
```

```
        print(node, end=" ")
```

```
        for neighbour in graph[node]:
```

```
            if neighbour not in visited:
```

```
                visited.append(neighbour)
```

```
                queue.append(neighbour)
```

```
print("BFS Traversal:")
```

```
bfs(graph, start_node)                # bfs(graph, start_node) → Performs BFS traversal.
```

Output:

BFS Traversal:

1 2 3 5 4 6

Q2) Write a program or report to compare Supervised Learning and Unsupervised Learning techniques. **[20 marks]**

Program to compare Supervised Learning and Unsupervised Learning

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

----- SUPERVISED LEARNING -----

Training data

Features: [Blood Sugar, Frequent Urination]

```
X = [
    [180, 1],
    [165, 1],
    [95, 0],
    [100, 0]
]
```

Target labels

1 = Diabetes

0 = No Diabetes

```
y = [1, 0, 0, 1]
```

Train the model

```
model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)
```

New patient

```
new_patient = [[175, 1]]
```

Predict

```
prediction = model.predict(new_patient)
```

```
print("SUPERVISED LEARNING")
```

```
if prediction[0] == 1:
    print("Prediction: Diabetes")
else:
    print("Prediction: No Diabetes")
```

----- UNSUPERVISED LEARNING -----

Data without target labels

```
data = [
    [180, 1],
    [165, 1],
    [95, 0],
    [100, 0]
]
```

Create K-Means model

```
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
```

```

# Train the model

kmeans.fit(data)

# Display clusters

print("\nUNSUPERVISED LEARNING")

print("Cluster Labels:", kmeans.labels_)

print("Cluster Centers:")
print(kmeans.cluster_centers_)

# ----- GRAPH -----

# Separate data according to clusters

cluster_0 = []
cluster_1 = []

for i in range(len(data)):
    if kmeans.labels_[i] == 0:
        cluster_0.append(data[i])
    else:
        cluster_1.append(data[i])

# Plot the clusters

plt.scatter(
    [x[0] for x in cluster_0],
    [x[1] for x in cluster_0],
    label="Cluster 0"
)

plt.scatter(
    [x[0] for x in cluster_1],
    [x[1] for x in cluster_1],
    label="Cluster 1"
)

# Plot cluster centers

plt.scatter(
    kmeans.cluster_centers_[0],
    kmeans.cluster_centers_[1],
    marker="X",
    s=200,
    label="Cluster Centers"
)

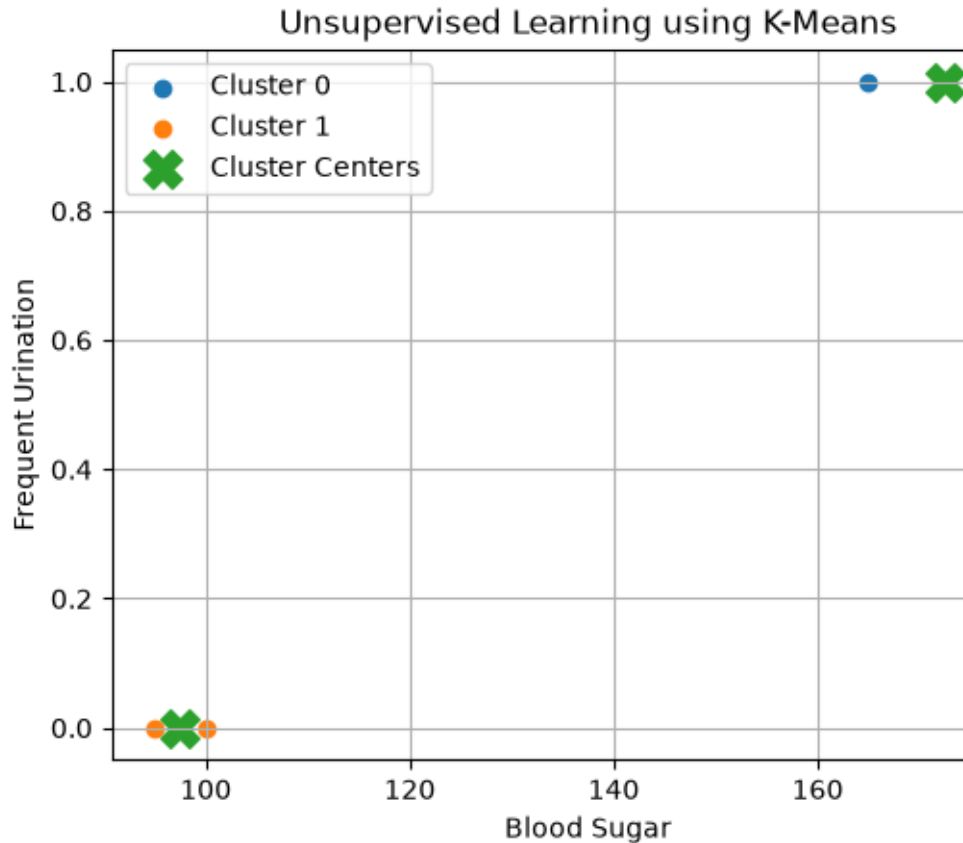
plt.xlabel("Blood Sugar")
plt.ylabel("Frequent Urination")
plt.title("Unsupervised Learning using K-Means")
plt.legend()
plt.grid(True)

plt.show()

```

Output:
SUPERVISED LEARNING
Prediction: Diabetes

UNSUPERVISED LEARNING
Cluster Labels: [0 0 1 1]
Cluster Centers:
[[172.5 1.]
[97.5 0.]]



OR

Q2) Write a program to demonstrate basic Ontological Engineering concepts [20 marks]
using classes, subclasses and relationships.

*# Program to demonstrate Basic Ontological Engineering
using classes, subclasses and relationships*

Classes

```
classes = {  
    "Animal": ["Dog", "Cat"],  
    "Food": ["Milk", "Meat"]  
}
```

Relationships

```
facts = [  
    ("Dog", "eats", "Meat"),  
    ("Cat", "eats", "Milk"),  
    ("Dog", "isA", "Animal"),  
    ("Cat", "isA", "Animal"),  
    ("Milk", "isA", "Food"),  
    ("Meat", "isA", "Food")  
]
```

Display classes and subclasses

```

print("Classes and Subclasses:")

for parent, children in classes.items():
    print(parent, "->", children)

# Display relationships

print("\nRelationships:")

for subject, relation, obj in facts:
    print(subject, relation, obj)

# Find subclass of Animal

animal = "Animal"

print("\nSubclasses of", animal, ":")

for parent, children in classes.items():
    if parent == animal:
        for child in children:
            print(child)

# Find food eaten by Dog

animal_name = "Dog"
food = ""

for subject, relation, obj in facts:
    if subject == animal_name and relation == "eats":
        food = obj

print("\nRelationship of", animal_name, ":")

if food != "":
    print(animal_name, "eats", food)
else:
    print("Relationship not found.")

# Ontology inference

if food != "":
    food_type = ""

    for subject, relation, obj in facts:
        if subject == food and relation == "isA":
            food_type = obj

    print("\nOntology Inference:")
    print(food, "is a", food_type)

```

Output:

Classes and Subclasses:
Animal -> ['Dog', 'Cat']
Food -> ['Milk', 'Meat']

Relationships:
Dog eats Meat
Cat eats Milk
Dog isA Animal
Cat isA Animal
Milk isA Food
Meat isA Food

Subclasses of Animal :

Dog

Cat

Relationship of Dog :

Dog eats Meat

Ontology Inference:

Meat is a Food

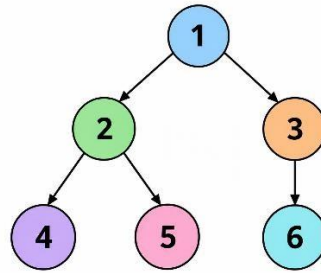
Q3) Viva

[05 marks]

Slip_16

Q1) Write a program to implement Depth Limited Search (DLS).

[10 marks]



Depth Limited Search (DLS) Program

Function to perform Depth Limited Search

```
def dls(graph, node, limit, path):
```

```
    # Add the current node to the path  
    path.append(node)
```

```
    # Stop searching if the depth limit is reached  
    if limit == 0:  
        return
```

```
    # Explore all neighbouring nodes  
    for neighbour in graph[node]:
```

```
        # Perform DLS recursively with reduced depth limit  
        dls(graph, neighbour, limit - 1, path)
```

State Space Representation of the Graph

```
graph = {  
    1: [2, 3],  
    2: [4, 5],  
    3: [6],  
    4: [],  
    5: [],  
    6: []  
}
```

Define the Initial State and Depth Limit

```
start = 1
```

```
depth_limit = 2
```

Create an empty path

```
path = []
```

Call the DLS Function

```
dls(graph, start, depth_limit, path)
```

Display the Result

```
print("Depth Limited Search Path:")
```

```
print(" -> ".join(map(str, path)))
```

Output:

Depth Limited Search Path:

1 -> 2 -> 4 -> 5 -> 3 -> 6

Q2) Write a program to implement the Backward Chaining inference mechanism. **[20 marks]**

```
# Program to implement Backward Chaining  
# Rule-Based Medical Diagnosis
```

```
# Input from user
```

```
temperature = float(input("Enter body temperature (°C): "))
```

```
cough = input("Does the patient have a cough? (yes/no): ").lower()
```

```
# Facts
```

```
if temperature > 38:
```

```
    fever = True
```

```
else:
```

```
    fever = False
```

```
# Goal: Patient has Flu
```

```
goal = "Flu"
```

```
# Backward Chaining
```

```
# To prove Flu, we need Fever AND Cough
```

```
if goal == "Flu":
```

```
    # Check first condition: Fever
```

```
    if fever:
```

```
        # Check second condition: Cough
```

```
        if cough == "yes":
```

```
            result = "Patient has Flu"
```

```
        else:
```

```
            result = "Flu cannot be proved because patient has no cough."
```

```
    else:
```

```
        result = "Flu cannot be proved because patient has no fever."
```

```
# Output
```

```
print("\n--- Backward Chaining Result ---")
```

```
print("Goal:", goal)
```

```
print("Fever:", fever)
```

```
print("Cough:", cough)
```

```
print("Result:", result)
```

```
--- Backward Chaining Result ---
```

```
Goal: Flu
```

```
Fever: False
```

```
Cough: no
```

```
Result: Flu cannot be proved because patient has no fever.
```

OR

- Q2)** Write a program to implement the K-Means Clustering Algorithm for grouping data into clusters. Use the data set provided in country_data.csv. The columns are: country nm, child_mortality, exports, health, imports, income, inflation, life_expectancy, total_fer, gdp. Normalize the data set for KMeans clustering (hint: use StandardScaler()). Apply KMeans with number of clusters = 3. Use different values for k from 1 to 10, use Elbow method to find the ideal value of 'k'. **[20 marks]**

```
Import :Country-data.csv
# Program to implement K-Means Clustering Algorithm

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Read the dataset

df = pd.read_csv("country-data.csv")

# Rename columns according to the question

df.columns = ['country nm', 'child_mortality', 'exports', 'health',
              'imports', 'income', 'inflation', 'life_expectancy',
              'total_fer', 'gdp']

# Display the dataset

print("Dataset:")
print(df)

# Select numerical columns for clustering

X = df[['child_mortality', 'exports', 'health', 'imports',
        'income', 'inflation', 'life_expectancy',
        'total_fer', 'gdp']]

# Normalize the dataset using StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\nNormalized Data:")
print(X_scaled)

# Apply K-Means with number of clusters = 3

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

kmeans.fit(X_scaled)

# Display cluster labels

print("\nCluster Labels:")
print(kmeans.labels_)

# Add cluster labels to the dataset

df['Cluster'] = kmeans.labels_

print("\nDataset with Cluster:")
print(df[['country nm', 'Cluster']])

# Elbow Method
```

```

wcss = []

for k in range(1, 11):

    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)

    kmeans.fit(X_scaled)

    wcss.append(kmeans.inertia_)

# Display WCSS values

print("\nWCSS Values:")

for k in range(1, 11):
    print("K =", k, " WCSS =", wcss[k - 1])

# Plot Elbow Graph

plt.plot(range(1, 11), wcss, marker='o')

plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method for Finding Optimal K")

plt.grid(True)
plt.show()

```

Output:

Dataset:

	country nm	child_mortality	exports	health	imports	income \
0	Afghanistan	90.2	10.0	7.58	44.9	1610
1	Albania	16.6	28.0	6.55	48.6	9930
2	Algeria	27.3	38.4	4.17	31.4	12900
3	Angola	119.0	62.3	2.85	42.9	5900
4	Antigua and Barbuda	10.3	45.5	6.03	58.9	19100
..	...	...	...	...	...	...
162	Vanuatu	29.2	46.6	5.25	52.7	2950
163	Venezuela	17.1	28.5	4.91	17.6	16500
164	Vietnam	23.3	72.0	6.84	80.2	4490
165	Yemen	56.3	30.0	5.18	34.4	4480
166	Zambia	83.1	37.0	5.89	30.9	3280

	inflation	life_expectancy	total_fer	gdp
0	9.44	56.2	5.82	553
1	4.49	76.3	1.65	4090
2	16.10	76.5	2.89	4460
3	22.40	60.1	6.16	3530
4	1.44	76.8	2.13	12200
..	...	...	...	...
162	2.62	63.0	3.50	2970
163	45.90	75.4	2.47	13500
164	12.10	73.1	1.95	1310
165	23.60	67.5	4.67	1310
166	14.00	52.0	5.40	1460

[167 rows x 10 columns]

Normalized Data:

```

[[ 1.29153238 -1.13827979  0.27908825 ... -1.61909203  1.90288227
 -0.67917961]
 [-0.5389489 -0.47965843 -0.09701618 ...  0.64786643 -0.85997281
 -0.48562324]
 [-0.27283273 -0.09912164 -0.96607302 ...  0.67042323 -0.0384044
 -0.46537561]
 ...
 [-0.37231541  1.13030491  0.0088773 ...  0.28695762 -0.66120626
 -0.63775406]

```

```
[ 0.44841668 -0.40647827 -0.59727159 ... -0.34463279 1.14094382
-0.63775406]
[ 1.11495062 -0.15034774 -0.33801514 ... -2.09278484 1.6246091
-0.62954556]]
```

Cluster Labels:

```
[1 2 2 1 2 2 2 0 0 2 2 0 2 2 2 0 2 1 2 2 2 1 2 0 2 1 1 2 1 0 2 1 1 2 2 2 1
1 1 2 1 2 0 0 0 2 2 2 2 2 1 1 2 2 0 0 1 1 2 0 1 0 2 2 1 1 2 1 2 0 2 2 2 1 0
0 0 2 0 2 2 1 1 0 2 1 2 2 1 1 2 2 0 2 1 1 2 2 1 0 1 2 2 2 2 2 2 1 2 1 2 0
0 1 1 0 2 1 2 2 2 2 2 0 0 2 2 1 2 2 1 2 2 1 0 0 0 2 1 0 0 2 2 1 2 0 0 2 1
2 1 1 2 2 2 2 1 2 0 0 0 2 2 2 2 2 1 1]
```

Dataset with Cluster:

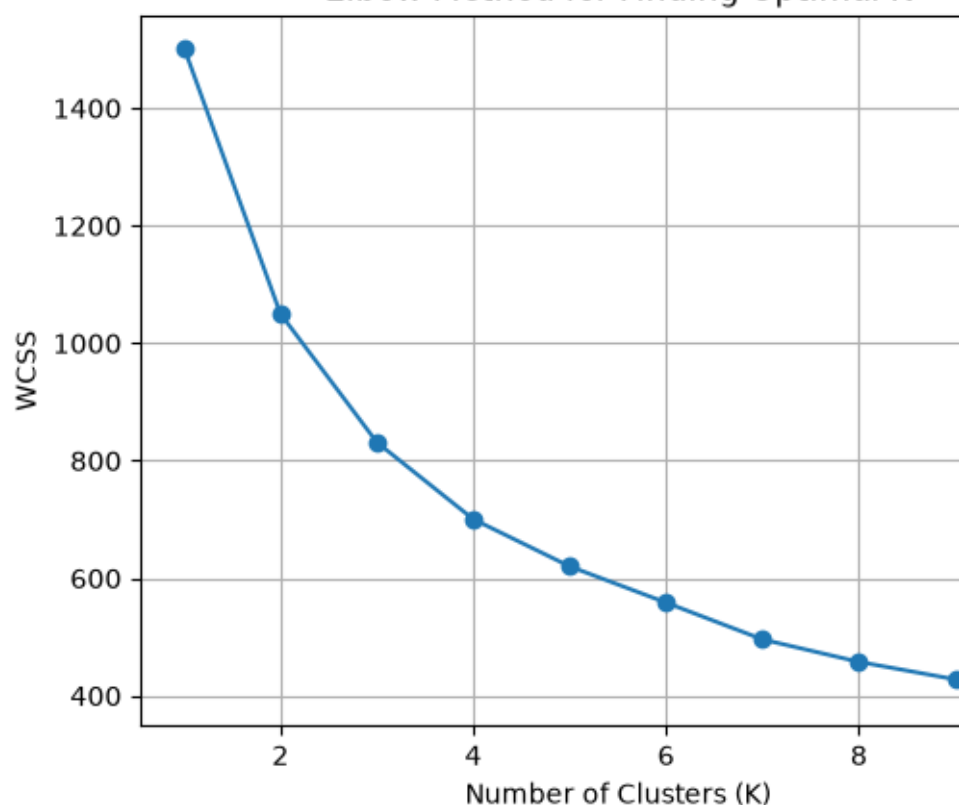
	country nm	Cluster
0	Afghanistan	1
1	Albania	2
2	Algeria	2
3	Angola	1
4	Antigua and Barbuda	2
..	...	...
162	Vanuatu	2
163	Venezuela	2
164	Vietnam	2
165	Yemen	1
166	Zambia	1

[167 rows x 2 columns]

WCSS Values:

```
K = 1 WCSS = 1503.0
K = 2 WCSS = 1050.2145582853304
K = 3 WCSS = 831.4244352086874
K = 4 WCSS = 700.5205369722203
K = 5 WCSS = 620.1633712888422
K = 6 WCSS = 558.4696598590399
K = 7 WCSS = 495.80794552153463
K = 8 WCSS = 457.58614789124226
K = 9 WCSS = 427.8036719813467
K = 10 WCSS = 403.2296125136436
```

Elbow Method for Finding Optimal K



Q3) Viva

[05 marks]

Slip_20

Q1) Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function.

[10 marks]

Hill Climbing Algorithm
Find the maximum value of a simple objective function

Define the heuristic values

```
heuristic = {  
    'A': 15,  
    'B': 25,  
    'D': 30,  
    'C': 35,  
    'E': 50  
}
```

Define the graph

```
graph = {  
    'A': ['B', 'D'],  
    'B': ['A', 'C', 'E'],  
    'D': ['A', 'C'],  
    'C': ['D', 'B', 'E'],  
    'E': ['C', 'B']  
}
```

Define the Hill Climbing function

```
def hill_climbing(start):
```

Set the starting state as the current state
current = start

Store the path
path = [current]

Repeat the process until no better neighbor is found
while True:

Generate neighboring states
neighbors = graph[current]

Select the neighbor having the highest value
best_neighbor = max(
 neighbors,
 key=**lambda** x: heuristic[x]
)

Check whether the best neighbor is better
than the current state
if heuristic[best_neighbor] **<=** heuristic[current]:

If there is no improvement, stop the algorithm
break

Move to the better neighboring state
current = best_neighbor

Add the state to the path
path.append(current)

Return the final state and path
return path, heuristic[current]

```

# Set the initial state
start = 'A'

# Call the Hill Climbing function
path, maximum_value = hill_climbing(start)

# Display the starting state
print("Starting State:", start)

# Display the path followed
print("Path:", " -> ".join(path))

# Display the state where maximum value is found
print("Maximum State:", path[-1])

# Display the maximum heuristic value
print("Maximum Value:", maximum_value)

```

Output:

Starting State: A
Path: A -> D -> C -> E
Maximum State: E
Maximum Value: 50

Q2) Write a program or report to compare Supervised Learning and Unsupervised Learning techniques. **[20 marks]**

Program to compare Supervised Learning and Unsupervised Learning

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

```

----- SUPERVISED LEARNING -----

Training data

Features: [Blood Sugar, Frequent Urination]

```

X = [
    [180, 1],
    [165, 1],
    [95, 0],
    [100, 0]
]

```

Target labels

1 = Diabetes

0 = No Diabetes

```

y = [1, 0, 0, 1]

```

Train the model

```

model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

```

New patient

```

new_patient = [[175, 1]]

```

```

# Predict

prediction = model.predict(new_patient)

print("SUPERVISED LEARNING")

if prediction[0] == 1:
    print("Prediction: Diabetes")
else:
    print("Prediction: No Diabetes")

# ----- UNSUPERVISED LEARNING -----

# Data without target labels

data = [
    [180, 1],
    [165, 1],
    [95, 0],
    [100, 0]
]

# Create K-Means model

kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)

# Train the model

kmeans.fit(data)

# Display clusters

print("\nUNSUPERVISED LEARNING")

print("Cluster Labels:", kmeans.labels_)

print("Cluster Centers:")
print(kmeans.cluster_centers_)

# ----- GRAPH -----

# Separate data according to clusters

cluster_0 = []
cluster_1 = []

for i in range(len(data)):
    if kmeans.labels_[i] == 0:
        cluster_0.append(data[i])
    else:
        cluster_1.append(data[i])

# Plot the clusters

plt.scatter(
    [x[0] for x in cluster_0],
    [x[1] for x in cluster_0],
    label="Cluster 0"

```

```

)

plt.scatter(
    [x[0] for x in cluster_1],
    [x[1] for x in cluster_1],
    label="Cluster 1"
)

# Plot cluster centers

plt.scatter(
    kmeans.cluster_centers_[0, 0],
    kmeans.cluster_centers_[0, 1],
    marker="X",
    s=200,
    label="Cluster Centers"
)

plt.xlabel("Blood Sugar")
plt.ylabel("Frequent Urination")
plt.title("Unsupervised Learning using K-Means")
plt.legend()
plt.grid(True)

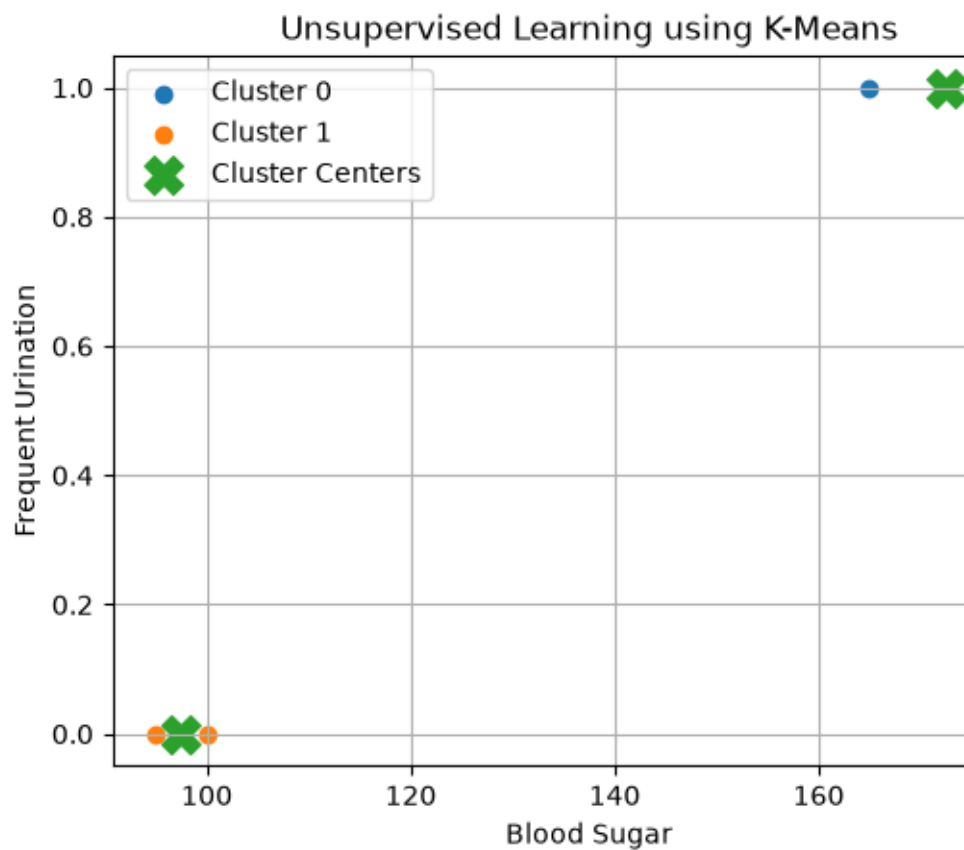
plt.show()

```

Output:

SUPERVISED LEARNING
Prediction: Diabetes

UNSUPERVISED LEARNING
Cluster Labels: [0 0 1 1]
Cluster Centers:
[[172.5 1.]
 [97.5 0.]]



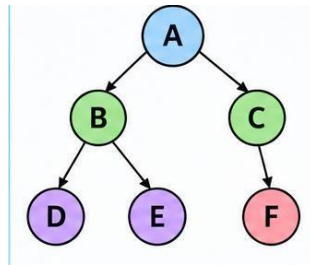
Q3) Viva

[05 marks]

Slip_21

Q1) Write a program implement Iterative Deepening Search (IDS).

[10 marks]



Iterative Deepening Search (IDS)

```
def dls(graph, node, goal, depth, visited):  
    if node == goal:  
        return True  
  
    if depth == 0:  
        return False  
  
    visited.add(node)  
  
    for neighbour in graph[node]:  
        if neighbour not in visited:  
            if dls(graph, neighbour, goal, depth - 1, visited):  
                return True  
  
    return False  
  
def ids(graph, start, goal, max_depth):  
    for depth in range(max_depth + 1):  
        visited = set()  
  
        if dls(graph, start, goal, depth, visited):  
            print("Goal found at depth", depth)  
            return True  
  
    print("Goal not found")  
    return False  
  
# Graph  
  
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': [],  
    'F': []  
}  
  
# Main program  
  
start = input("Enter start node: ")
```

```

goal = input("Enter goal node: ")

max_depth = int(input("Enter maximum depth: "))

ids(graph, start, goal, max_depth)
Output:
Goal not found
False

```

Q2) Write a program to implement the Gaussian Naïve Bayes Classifier for classifying numerical data. [20 marks]

*# Program to implement Gaussian Naive Bayes Classifier
for classifying numerical data*

```

from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

```

*# Training data
Features: [Age, Income]*

```

X = [
    [25, 20000],
    [30, 25000],
    [35, 30000],
    [40, 40000],
    [45, 50000],
    [50, 60000],
    [28, 22000],
    [38, 35000],
    [48, 55000],
    [55, 65000]
]

```

*# Target values
0 = Loan Rejected
1 = Loan Approved*

```

y = [0, 0, 0, 1, 1, 1, 0, 1, 1, 1]

```

Split data into training and testing data

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

Create Gaussian Naive Bayes model

```

model = GaussianNB()

```

Train the model

```

model.fit(X_train, y_train)

```

Predict test data

```

y_prediction = model.predict(X_test)

```

Calculate accuracy

```

accuracy = accuracy_score(y_test, y_prediction)

```

```

print("Gaussian Naive Bayes Classifier")
print("-----")

```

```

print("Test Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")

```

Test a new numerical data

```

test_data = [[42, 45000]]

```

```

prediction = model.predict(test_data)

```

```
probability = model.predict_proba(test_data)
```

```
print("\nTest Data:")  
print("Age:", test_data[0][0])  
print("Income:", test_data[0][1])
```

```
print("\nPrediction:")
```

```
if prediction[0] == 1:  
    print("Loan Approved")  
else:  
    print("Loan Rejected")
```

```
print("\nProbability of Loan Rejection:",  
      round(probability[0][0], 2))
```

```
print("Probability of Loan Approval:",  
      round(probability[0][1], 2))
```

Output:

Gaussian Naive Bayes Classifier

Test Accuracy: 1.0
Accuracy Percentage: 100.0 %

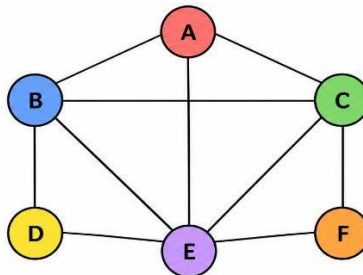
Test Data:
Age: 42
Income: 45000

Prediction:
Loan Approved

Probability of Loan Rejection: 0.0
Probability of Loan Approval: 1.0

OR

- Q2)** Write a program to solve the Map Colouring Problem using the **[20 marks]** Constraint Satisfaction Problem (CSP) approach.



```
# Program to solve Map Coloring Problem using CSP  
# Using Backtracking
```

```
# Function to check whether color can be assigned  
def is_safe(vertex, graph, color, c):
```

```
    # Check adjacent vertices
```

```
    for i in range(6):  
        if graph[vertex][i] == 1 and color[i] == c:  
            return False
```

```
    # Make sure the color is not already used
```

```
    if c in color:
```

```

        return False

    return True

# Backtracking function
def graph_coloring(graph, color, vertex):

    # If all vertices are colored
    if vertex == 6:
        return True

    # Try different colors
    for c in range(1, 7):

        # Check whether color is safe
        if is_safe(vertex, graph, color, c):

            # Assign color
            color[vertex] = c

            # Move to next vertex
            if graph_coloring(graph, color, vertex + 1):
                return True

            # Backtrack
            color[vertex] = 0

    return False

# ----- Main Program -----

# Graph connections:
# A -> B, C, E
# B -> A, C, D, E
# C -> A, B, E, F
# D -> B, E
# E -> A, B, C, D, F
# F -> C, E

graph = [
    [0, 1, 1, 0, 1, 0], # A
    [1, 0, 1, 1, 1, 0], # B
    [1, 1, 0, 0, 1, 1], # C
    [0, 1, 0, 0, 1, 0], # D
    [1, 1, 1, 1, 0, 1], # E
    [0, 0, 1, 0, 1, 0] # F
]

# Number of colors
m = 6

# 0 means no color assigned
color = [0, 0, 0, 0, 0, 0]

# Color names
color_names = [
    "",
    "Red",
    "Blue",
    "Green",
    "Yellow",
    "Purple",
    "Orange"
]

# Apply graph coloring

```

```
if graph_coloring(graph, color, 0):  
    print("Map Coloring Solution:")  
    for i in range(6):  
        print(chr(65 + i), "->", color_names[color[i]])  
else:  
    print("No solution exists with", m, "colors.")  
Output:  
Map Coloring Solution:  
A -> Red  
B -> Blue  
C -> Green  
D -> Yellow  
E -> Purple  
F -> Orange
```

Q3) Viva

[05 marks]

Slip_22

Q1) Write a program to solve the Tower of Hanoi Problem using state-space representation. **[10 marks]**

*# Program to solve Tower of Hanoi Problem
using State-Space Representation*

```
def tower_of_hanoi(n, source, auxiliary, destination):  
  
    if n == 1:  
        print("Move disk 1 from", source, "to", destination)  
        return  
  
    tower_of_hanoi(n - 1, source, destination, auxiliary)  
  
    print("Move disk", n, "from", source, "to", destination)  
  
    tower_of_hanoi(n - 1, auxiliary, source, destination)
```

Main program

```
n = int(input("Enter number of disks: "))
```

```
print("\nInitial State:")  
print("A =", list(range(n, 0, -1)))  
print("B = []")  
print("C = []")
```

```
print("\nMoves:")
```

```
tower_of_hanoi(n, 'A', 'B', 'C')
```

```
print("\nGoal State:")  
print("A = []")  
print("B = []")  
print("C =", list(range(n, 0, -1)))
```

Output:

Initial State:

A = [3, 2, 1]

B = []

C = []

Moves:

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from C to B

Move disk 3 from A to C

Move disk 1 from B to A

Move disk 2 from B to C

Move disk 1 from A to C

Goal State:

A = []

B = []

C = [3, 2, 1]

Q2) Write a program to solve the N-Queens Problem using the Backtracking Algorithm. **[20 marks]**

Program of N-Queens using Backtracking

Function to check if placing a queen at row and column is safe

```
def is_safe(board, row, col, n):
```

```
    # Check the column
```

```
    for i in range(row):
```

```
        if board[i][col] == 1:
```

```
            return False
```

```
    # Check the upper-left diagonal
```

```
    i = row - 1
```

```
    j = col - 1
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j] == 1:
```

```
            return False
```

```
        i = i - 1
```

```
        j = j - 1
```

```
    # Check the upper-right diagonal
```

```
    i = row - 1
```

```
    j = col + 1
```

```
    while i >= 0 and j < n:
```

```
        if board[i][j] == 1:
```

```
            return False
```

```
        i = i - 1
```

```
        j = j + 1
```

```
    return True
```

```
# Backtracking function
```

```
def solve_n_queens(board, n, row):
```

```
    # If all queens are placed
```

```
    if row == n:
```

```
        return True
```

```
    # Try different columns
```

```
    for col in range(n):
```

```
        # Check if placing queen is safe
```

```
        if is_safe(board, row, col, n):
```

```
            # Place the queen
```

```
            board[row][col] = 1
```

```
            # Move to the next row
```

```
            if solve_n_queens(board, n, row + 1):
```

```
                return True
```

```
            # Backtrack
```

```
            board[row][col] = 0
```

```
    return False
```

```
# ----- Main Program -----
```

```
# Number of queens
```

```
n = 4
```

```

# Create an empty chessboard
# 0 means empty and 1 means queen
board = [[0] * n for _ in range(n)]

# Call the backtracking function
if solve_n_queens(board, n, 0):

    # Display the solution
    print("N-Queens Solution:")

    # Display the chessboard
    for i in range(n):
        print(board[i])

else:
    print("No solution exists.")

```

Output:
N-Queens Solution:
[0, 1, 0, 0]
[0, 0, 0, 1]
[1, 0, 0, 0]
[0, 0, 1, 0]

OR

Q2) Write a program to design a Rule-Based Expert System for simple decision-making. **[20 marks]**

Program of Rule-Based Expert System for Student Result

```

# Input from user
marks = float(input("Enter marks: "))
attendance = float(input("Enter attendance percentage: "))

# Rule 1: Check whether student has passed
if marks >= 40:
    passed = True
else:
    passed = False

# Rule 2: Check attendance
if attendance >= 75:
    good_attendance = True
else:
    good_attendance = False

# Rule 3 and Rule 4: Decision and recommendation
if passed and good_attendance:
    result = "Eligible for Exam"
    recommendation = "Student has good marks and attendance."
elif passed and not good_attendance:
    result = "Attendance Shortage"
    recommendation = "Improve attendance before the examination."
elif not passed and good_attendance:
    result = "Failed"
    recommendation = "Improve marks and prepare better."
else:
    result = "Not Eligible"
    recommendation = "Improve marks and attendance."

# Output
print("\n--- Student Report ---")
print("Passed:", passed)
print("Good Attendance:", good_attendance)

```

```
print("Decision:", result)
print("Recommendation:", recommendation)
```

Output:

--- Student Report ---

Passed: True

Good Attendance: True

Decision: Eligible for Exam

Recommendation: Student has good marks and attendance.

Q3) Viva

[05 marks]

Slip_23

Q1) Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function. ($F(x) = -x^2 + 4x$) **[10 marks]**

Hill Climbing Algorithm

Find the maximum value of $F(x) = -x^2 + 4x$

Define the objective function

```
def f(x):  
    return -x * x + 4 * x
```

Hill Climbing function

```
def hill_climbing(start):
```

```
    current = start
```

```
    path = [current]
```

```
    while True:
```

```
        # Generate neighboring states
```

```
        left = current - 1
```

```
        right = current + 1
```

```
        # Find the neighbor with the highest value
```

```
        if f(left) > f(right):
```

```
            best_neighbor = left
```

```
        else:
```

```
            best_neighbor = right
```

```
        # Check whether the neighbor is better
```

```
        if f(best_neighbor) <= f(current):
```

```
            break
```

```
        # Move to the better neighbor
```

```
        current = best_neighbor
```

```
        path.append(current)
```

```
    return path, current, f(current)
```

Main program

```
start = int(input("Enter starting value of x: "))
```

```
path, maximum_x, maximum_value = hill_climbing(start)
```

```
print("\nPath:", " -> ".join(map(str, path)))
```

```
print("Maximum x:", maximum_x)
```

```
print("Maximum value:", maximum_value)
```

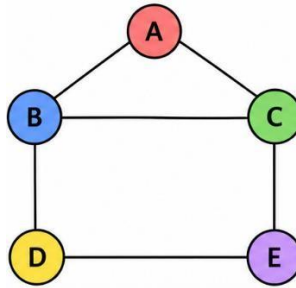
Output:

Path: 2

Maximum x: 2

Maximum value: 4

Q2) Write a program to solve the Map Colouring Problem using the **[20 marks]** Constraint Satisfaction Problem (CSP) approach.



*# Program to solve Map Coloring Problem using CSP
Using Backtracking*

Function to check whether color can be assigned

```
def is_safe(vertex, graph, color, c):
```

Check adjacent vertices

```
for i in range(5):
    if graph[vertex][i] == 1 and color[i] == c:
        return False
```

Make sure the color is not already used

```
if c in color:
    return False
```

```
return True
```

Backtracking function

```
def graph_coloring(graph, color, vertex):
```

If all vertices are colored

```
if vertex == 5:
    return True
```

Try different colors

```
for c in range(1, 6):
```

Check whether color is safe

```
if is_safe(vertex, graph, color, c):
```

Assign color

```
color[vertex] = c
```

Move to next vertex

```
if graph_coloring(graph, color, vertex + 1):
    return True
```

Backtrack

```
color[vertex] = 0
```

```
return False
```

----- Main Program -----

Graph connections:

A -> B, C

B -> A, C, D

C -> A, B, E

D -> B, E

E -> C, D

```
graph = [
```

```

    [0, 1, 1, 0, 0], # A
    [1, 0, 1, 1, 0], # B
    [1, 1, 0, 0, 1], # C
    [0, 1, 0, 0, 1], # D
    [0, 0, 1, 1, 0] # E
]

# Number of colors
m = 5

# 0 means no color assigned
color = [0, 0, 0, 0, 0]

# Color names
color_names = [
    "",
    "Red",
    "Blue",
    "Green",
    "Yellow",
    "Purple"
]

# Apply graph coloring
if graph_coloring(graph, color, 0):

    print("Map Coloring Solution:")

    for i in range(5):
        print(chr(65 + i), "->", color_names[color[i]])

else:
    print("No solution exists with", m, "colors.")

```

Output:

Map Coloring Solution:

A -> Red

B -> Blue

C -> Green

D -> Yellow

E -> Purple

OR

Q2) Write a program or report to compare Supervised Learning and Unsupervised Learning techniques. **[20 marks]**

Program to compare Supervised Learning and Unsupervised Learning

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

```

----- SUPERVISED LEARNING -----

Training data

Features: [Blood Sugar, Frequent Urination]

```

X = [
    [180, 1],
    [165, 1],
    [95, 0],
    [100, 0]
]

```

```

# Target labels
# 1 = Diabetes
# 0 = No Diabetes

y = [1, 0, 0, 1]

# Train the model

model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

# New patient

new_patient = [[175, 1]]

# Predict

prediction = model.predict(new_patient)

print("SUPERVISED LEARNING")

if prediction[0] == 1:
    print("Prediction: Diabetes")
else:
    print("Prediction: No Diabetes")

# ----- UNSUPERVISED LEARNING -----

# Data without target labels

data = [
    [180, 1],
    [165, 1],
    [95, 0],
    [100, 0]
]

# Create K-Means model

kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)

# Train the model

kmeans.fit(data)

# Display clusters

print("\nUNSUPERVISED LEARNING")

print("Cluster Labels:", kmeans.labels_)

print("Cluster Centers:")
print(kmeans.cluster_centers_)

# ----- GRAPH -----

# Separate data according to clusters

```

```

cluster_0 = []
cluster_1 = []

for i in range(len(data)):

    if kmeans.labels_[i] == 0:
        cluster_0.append(data[i])
    else:
        cluster_1.append(data[i])

# Plot the clusters

plt.scatter(
    [x[0] for x in cluster_0],
    [x[1] for x in cluster_0],
    label="Cluster 0"
)

plt.scatter(
    [x[0] for x in cluster_1],
    [x[1] for x in cluster_1],
    label="Cluster 1"
)

# Plot cluster centers

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker="X",
    s=200,
    label="Cluster Centers"
)

plt.xlabel("Blood Sugar")
plt.ylabel("Frequent Urination")
plt.title("Unsupervised Learning using K-Means")
plt.legend()
plt.grid(True)

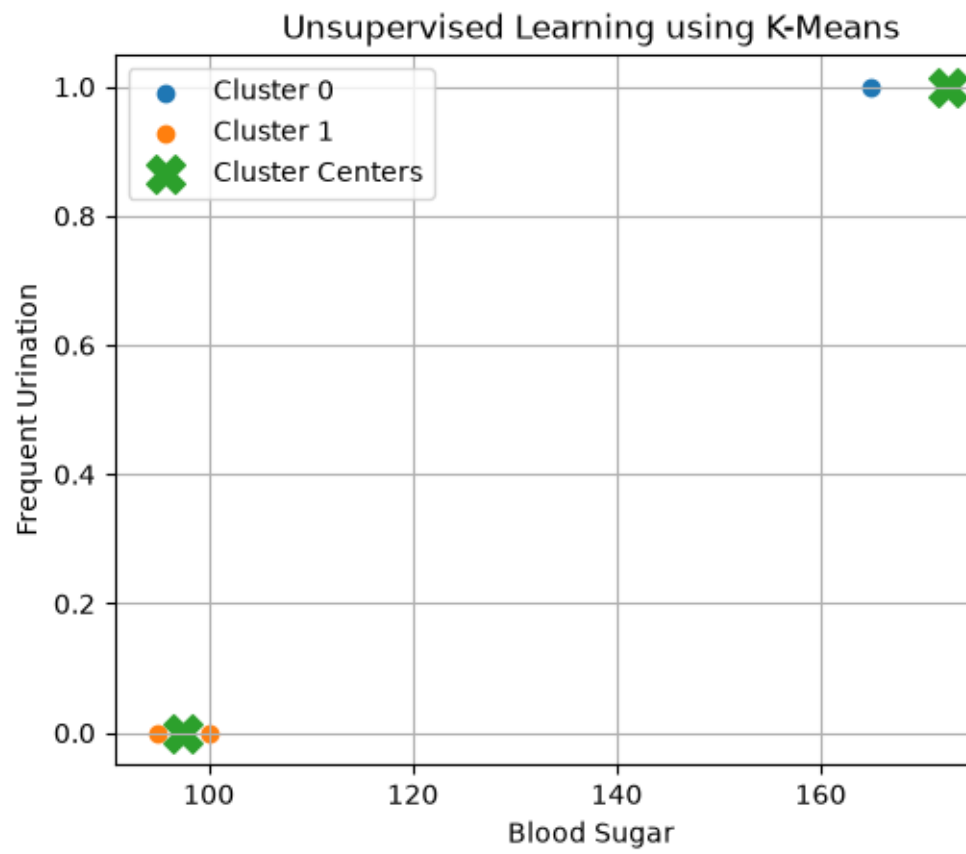
plt.show()

```

Output:

SUPERVISED LEARNING
Prediction: Diabetes

UNSUPERVISED LEARNING
Cluster Labels: [0 0 1 1]
Cluster Centers:
[[172.5 1.]
 [97.5 0.]]



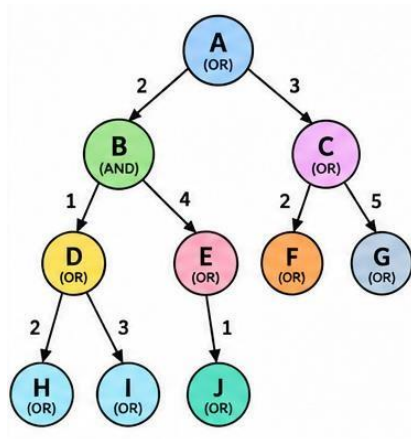
Q3) Viva

[05 marks]

Slip_25

Q1) Write the program to implement the AO* Algorithm

[10 marks]



To implement AO* (AND-OR Star) Algorithm
for solving an AND-OR graph.

AND-OR graph

```
graph = {
    'A': [('B', 'C')],
    'B': [('D', 'E')],
    'C': [('F', 'G')],
    'D': [('H', 'I')],
    'E': [('J')],
    'F': [],
    'G': [],
    'H': [],
    'I': [],
    'J': []
}
```

Node types

```
node_type = {
    'A': 'OR',
    'B': 'AND',
    'C': 'OR',
    'D': 'OR',
    'E': 'OR',
    'F': 'OR',
    'G': 'OR',
    'H': 'OR',
    'I': 'OR',
    'J': 'OR'
}
```

Distance / cost between connected nodes

```
distance = {
    ('A', 'B'): 2,
    ('A', 'C'): 3,
    ('B', 'D'): 1,
    ('B', 'E'): 4,
    ('D', 'H'): 2,
    ('D', 'I'): 3,
    ('E', 'J'): 1,
    ('C', 'F'): 2,
    ('C', 'G'): 5
}
```

```
def ao_star(node):
```

```

# OR node with no children is a goal/leaf node
if len(graph[node]) == 0:
    return 0

print("\nChecking Node:", node)

children = graph[node][0]

if node_type[node] == 'AND':

    # AND node: solve ALL children
    cost = 0

    for child in children:
        child_cost = ao_star(child)
        cost = cost + distance[(node, child)] + child_cost

    print("AND Node - Solve all children:", children)
    print("Cost(", node, ") =", cost)

    return cost

else:

    # OR node: choose ONE child with minimum cost
    costs = {}

    for child in children:
        child_cost = ao_star(child)
        cost = distance[(node, child)] + child_cost
        costs[child] = cost

    best_child = min(costs, key=costs.get)
    best_cost = costs[best_child]

    print("OR Node - Choose minimum cost child")
    print("Costs:", costs)
    print("Choose:", best_child)
    print("Cost(", node, ") =", best_cost)

    return best_cost

# Driver code
start_node = 'A'

print("AO* (AND-OR STAR) ALGORITHM")
print("-----")

cost = ao_star(start_node)

print("\nOptimal Solution:")
print("A -> C -> F")

print("Total Minimum Cost =", cost)

Output:
AO* (AND-OR STAR) ALGORITHM
-----

Checking Node: A

Checking Node: B

Checking Node: D
OR Node - Choose minimum cost child
Costs: {'H': 2, 'I': 3}
Choose: H

```

Cost(D) = 2

Checking Node: E

OR Node - Choose minimum cost child

Costs: { 'J': 1 }

Choose: J

Cost(E) = 1

AND Node - Solve all children: ('D', 'E')

Cost(B) = 8

Checking Node: C

OR Node - Choose minimum cost child

Costs: { 'F': 2, 'G': 5 }

Choose: F

Cost(C) = 2

OR Node - Choose minimum cost child

Costs: { 'B': 10, 'C': 5 }

Choose: C

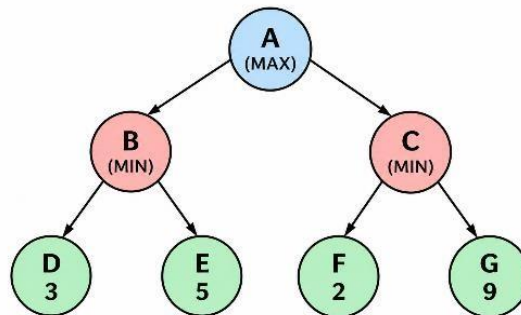
Cost(A) = 5

Optimal Solution:

A -> C -> F

Total Minimum Cost = 5

Q2) Write a program to implement the Minimax Algorithm for a two-player game **[20 marks]**
game.



Program for .

```
import math
```

```
# Recursive Minimax function
```

```
def minimax(depth, node_index, is_max, values, max_depth):
```

```
# If leaf node is reached
```

```
if depth == max_depth:
```

```
    return values[node_index]
```

```
if is_max:
```

```
# MAX player's turn
```

```
left = minimax(
```

```
    depth + 1,
```

```
    node_index * 2,
```

```
    False,
```

```
    values,
```

```
    max_depth
```

```
)
```

```
right = minimax(
```

```
    depth + 1,
```

```
    node_index * 2 + 1,
```

```
    False,
```

```
    values,
```

```

        max_depth
    )

    return max(left, right)

else:

    # MIN player's turn
    left = minimax(
        depth + 1,
        node_index * 2,
        True,
        values,
        max_depth
    )

    right = minimax(
        depth + 1,
        node_index * 2 + 1,
        True,
        values,
        max_depth
    )

    return min(left, right)

# ----- Main Program -----

# Leaf node values
# D = 3, E = 5, F = 2, G = 9
leaf_values = [3, 5, 2, 9]

# Height of tree
tree_depth = int(math.log2(len(leaf_values)))

# Find the optimal value
best_score = minimax(
    depth=0,
    node_index=0,
    is_max=True,
    values=leaf_values,
    max_depth=tree_depth
)

print("Leaf Node Values:", leaf_values)
print("Optimal value selected by Minimax:", best_score)
Output:
Leaf Node Values: [3, 5, 2, 9]
Optimal value selected by Minimax: 3

```

OR

Q2) Write a program to implement the Forward Chaining inference mechanism. **[20 marks]**

Program to implement Forward Chaining Inference Mechanism

```

# Known facts
facts = ["Ravi is a Student"]

# IF-THEN Rules
rules = [
    ("Ravi is a Student", "Ravi studies AI"),
    ("Ravi studies AI", "Ravi learns Machine Learning"),
    ("Ravi learns Machine Learning", "Ravi can get a Job")
]

```

```

]

# Desired goal
goal = "Ravi can get a Job"

print("Initial Facts:", facts)

# Forward Chaining
while True:

    rule_fired = False

    # Compare facts with rules
    for condition, conclusion in rules:

        # IF condition is true
        if condition in facts and conclusion not in facts:

            # THEN add new fact
            facts.append(conclusion)

            print("New Fact:", conclusion)

            rule_fired = True

        # Stop when goal is reached
        if conclusion == goal:
            break

    # Check whether goal is reached
    if goal in facts:
        break

    # Stop if no rule can be applied
    if rule_fired == False:
        break

# Display final result
print("\nFinal Facts:", facts)

if goal in facts:
    print("Goal Reached:", goal)
else:
    print("Goal Not Reached")

```

Output:

Initial Facts: ['Ravi is a Student']
 New Fact: Ravi studies AI
 New Fact: Ravi learns Machine Learning
 New Fact: Ravi can get a Job

Final Facts: ['Ravi is a Student', 'Ravi studies AI', 'Ravi learns Machine Learning', 'Ravi can get a Job']
 Goal Reached: Ravi can get a Job

Q3) Viva

[05 marks]